Meowtion

On-device behaviour recognition for early feline health insight

Jerome Graves and Rose Delcour-Min

Revision 3.0 • June 2026

Abstract. Cats are experts at hiding illness and pain. By the time something looks obviously wrong it is often already advanced, because the first warning is rarely a dramatic symptom; it's a shift in routine: eating a little less, drinking a little more, grooming and moving less than before. No owner can watch for that continuously, and memory makes a poor baseline. Meowtion turns the collar itself into that baseline. A lightweight, battery-powered collar classifies activities on the device in real time with motion and audio capture, using a confidence-gated cascade of two quantised neural networks. A motion model runs continuously, and only when movement alone is ambiguous does a short audio model step in to confirm. Raw audio never leaves the collar; only the result is relayed, over Bluetooth Low Energy through a home base station, to a live dashboard that surfaces the habit changes that matter for health. Critically, the activities are not fixed in firmware: owners define what to recognise, and a cloud trainer delivers new models back to the collar over the air. The outcome is a complete system, demonstrated end to end on real hardware, and a blueprint for private, on-device health monitoring that is a programmable platform, not a single-purpose gadget.

Acknowledgements

Meowtion is the work of **Jerome Graves** and **Rose Delcour-Min**. It stands on a large body of open-source work, the Zephyr and ESP-IDF projects, TensorFlow Lite Micro and CMSIS-NN, Firebase and Streamlit, and the wider embedded and machine-learning communities whose tools made a two-person build of this scope possible.

Our greatest debt, though, is to **Rupert**, the cat. He wore every awkward prototype without complaint, treated each new collar as a minor inconvenience between naps, and, simply by being himself, supplied the ground truth this entire system is built on. Every eating, drinking, grooming and resting example in these pages traces back to him. He remains entirely unimpressed.

Contents

List of Figures

List of Tables

Glossary of terms

A reader does not need to know every term below to follow the document; this list is here to look one up when it appears. Concepts are explained in plain language; acronyms are expanded.

ATT / GATT
: The Bluetooth Low Energy layers that define "characteristics" (named values a device exposes) and the services that group them.

BLE (Bluetooth Low Energy)
: A short-range, low-power wireless protocol; the collar's only radio.

BMS
: Battery Management System: the protection circuit built into the LiPo cell.

Confidence-gated cascade
: The collar's two-model scheme: a motion model runs on every window, and a second audio model is consulted only when the motion model is unsure.

CMSIS-NN
: ARM's hand-optimised neural-network routines for Cortex-M microcontrollers.

CRC
: Cyclic Redundancy Check: a checksum used to verify an over-the-air model transfer.

ESP-IDF
: Espressif's official development framework (SDK) for the ESP32 family; the station's software stack.

Firebase
: Google's hosted backend platform: user sign-in, a database, file storage, and serverless functions, used here as the cloud tier.

Firmware
: The program that runs on a microcontroller (the collar and the station each run their own).

IMU (Inertial Measurement Unit)
: The collar's six-axis motion sensor: a three-axis accelerometer plus a three-axis gyroscope.

Inference
: Running a trained model on new input to get a prediction (here, classifying a behaviour on the collar).

int8 quantisation
: Storing a model's numbers as 8-bit integers instead of floating point, which shrinks it and speeds it up enough to run on a microcontroller.

JWT
: JSON Web Token: the signed token format of a Firebase login.

MTU
: Maximum Transmission Unit: the negotiated size of a BLE data packet (247 bytes here).

NCS (nRF Connect SDK)
: Nordic's distribution of the Zephyr operating system, used to build the collar firmware.

NimBLE
: The lightweight Bluetooth stack used in the station firmware.

NVS
Non-Volatile Storage: the station's small flash key/value store for its credentials.

OTA (Over The Air)
Delivering a new trained model to the collar wirelessly over Bluetooth, with no cable or reflash.

PDM
Pulse-Density Modulation: the raw output format of the collar's MEMS microphone.

PHY
The Bluetooth physical layer; the collar uses the 2 Mbit variant for speed.

RSSI
Received Signal Strength Indication: how strong the collar's signal is at the station, used as a "close enough to record" gate.

RTDB (Realtime Database)
Firebase's live, JSON-tree database, holding each cat's current state and history.

SoC (System on Chip)
A single chip combining processor, memory and radio , the nRF52840 and ESP32-S3 modules.

Tensor arena
A fixed block of RAM the inference engine uses as scratch space for one model.

TFLM (TensorFlow Lite Micro)
The on-device engine that runs the trained models on the collar.

UF2
The drag-and-drop file format used to flash the collar over USB.

Web Serial
A browser feature that lets a web page talk to a USB device, used to pair the station.

Zephyr
A real-time operating system for microcontrollers; the base of the collar firmware.

μ-law
An 8-bit audio compression scheme (G.711) that halves the audio sent over Bluetooth.

PART I

Overview

1 Introduction

1.1 Motivation

Cats evolved to conceal illness. A cat that feels unwell masks it for as long as it can, so by the time a problem is obvious to an owner it is often advanced [1, 2]. The early signal is rarely a dramatic symptom; it is a quiet shift in routine. A cat that drinks noticeably more, eats less, stops grooming, or sleeps through activity it used to enjoy is communicating a change. No person watches their cat continuously, and day-to-day memory is a poor baseline, so the change stays invisible.

Meowtion makes that change visible. It observes the behaviours that make up a cat's routine, continuously and unobtrusively, and surfaces the trend to the owner. The goal is *not* to diagnose, but to notice a routine change early enough that the owner can act on it. The feline medicine literature is consistent that appetite, thirst, grooming and activity are among the first things to shift when a cat becomes unwell [1, 3], and that these shifts are easy to miss without a continuous, objective baseline of the kind a wearable activity monitor can provide [4].

1.2 Approach

The system is a wearable plus a small fixed gateway. A lightweight, battery-powered collar carries the sensors and the intelligence: it classifies behaviour on-device, in real time, and never streams raw sensor data off the cat. It reports only a compact result (for example, "eating, high confidence") over Bluetooth Low Energy (BLE) to a mains-powered station in the home. The station bridges BLE to the internet over Wi-Fi and forwards results to a cloud backend, which drives a hosted dashboard where the owner sees their cat's activity and trends live, with longer-run habit changes flagged automatically.

Figure 1: A cat wearing the Meowtion collar.

1.3 Design principles

Four commitments shape the whole system and recur throughout this document.

Privacy by construction.
Classification happens on the collar. The microphone only helps recognise behaviour on-device; raw audio is processed and discarded, never recorded or transmitted in normal operation. Each owner's data is scoped to their own account by the backend's security rules (section 13).

Trainable, not fixed-function.
The owner defines the recognised behaviours; they are not baked into firmware. Owners capture and label example clips, a cloud trainer produces tiny models, and those models are delivered to the collar over the air. Meowtion is a platform for recognising any pet behaviour, not a three-trick gadget (section 8).

Intelligence at the edge.
Inference runs on a microcontroller on the cat, not in the cloud. This keeps the radio uplink tiny (a classification result, not a sensor stream), keeps average power low enough for a collar-sized battery, and removes the cloud from the sensing loop (section 7).

Health-first analytics.
The dashboard is built around longitudinal habit tracking and baseline comparison, not just a live read-out, so a sustained change in a key habit surfaces as an insight rather than buried in raw events (section 10).

1.4 Contributions

As an engineering artefact, Meowtion's notable elements are: a *three-tier, confidence-gated cascade* in which a rules-based activity gate rests the collar in low power while the cat is still, a motion classifier runs while it is active, and a short audio classifier is invoked only to resolve ambiguous behaviours, so audio is a deployed disambiguation signal rather than only a training aid and power tracks how active the cat actually is (section 7); a *label-agnostic* pipeline in which the behaviour set is data-defined end to end, from the dashboard's action list through the cloud trainer to the index-based on-collar classifier; an *over-the-air model path* that delivers quantised models to a Bluetooth-only wearable through the station; and a privacy and multi-tenancy model in which battery devices hold only a scoped, revocable token and never the owner's credentials.

1.5 Scope and structure

This document is the complete technical reference for Meowtion as built. A reader can follow it top-to-bottom or jump to one tier. Section 3 gives the system-level picture, the component responsibilities, the two operating modes, and the end-to-end data flows, and is the right place to start. The detailed chapters then proceed bottom-up: the physical hardware (section 4); the firmware on the two microcontrollers (sections 5 and 6); the on-device AI (section 7); the training pipeline (section 8); the cloud backend (section 9); the dashboard (section 10); the communication protocols (section 12); and the security and privacy model (section 13). Section 15 records known limitations and planned work.

2 Requirements and design constraints

This section states the requirements the system meets and the constraints that shaped it, so the chapters that follow read as the response to them. The final column of each table points to the section where the requirement is addressed.

2.1 Functional requirements

Table 1 lists what the system must do for an owner.

Table 1: Functional requirements.

ID	Requirement	Addressed in
FR-1	Recognise a cat's everyday behaviours (eating, drinking, activity, rest, grooming) from a collar-worn sensor.	section 7
FR-2	Perform classification on the wearable itself, in real time, without streaming raw sensor data off the cat.	section 7
FR-3	Let the owner define the behaviour set; do not hard-code it in firmware.	section 8
FR-4	Provide a closed training loop (capture, label, train, deploy) driven entirely from the dashboard, with no local toolchain.	section 8
FR-5	Deliver trained models to the collar over the air, with no firmware reflash.	sections 6 and 12
FR-6	Present the owner a live view and a longitudinal history with health-oriented analytics.	section 10
FR-7	Support multiple owners, each seeing only their own cats and devices.	section 9
FR-8	Offer a public, read-only way to explore the product without an account.	section 10

2.2 Non-functional requirements

Table 2 lists the quality attributes and limits the design must meet, the constraints behind many of the engineering choices that follow.

Table 2: Non-functional requirements and quality attributes.

ID	Requirement	Addressed in
NFR-1	**Power.** The collar must run for a useful time on a collar-sized cell, which precludes Wi-Fi on the wearable, bounds the duty cycle of the audio path, and is met chiefly by the rules-based activity gate that rests the collar between bouts of motion.	sections 4, 5 and 7
NFR-2	**Footprint.** Inference must fit the microcontroller's RAM alongside the BLE stack.	section 7
NFR-3	**Privacy.** Raw audio must never leave the collar in normal operation; data must be scoped per owner.	section 13
NFR-4	**Credential safety.** Battery devices must not hold the owner's password; device authorisation must be revocable.	section 13
NFR-5	**Reproducibility.** A third party must be able to build and run the system from the repository without contacting the authors.	section 9
NFR-6	**Openness.** Hardware designs and code are released under open licences.	section 4

2.3 Constraints

Radio.

The collar is Bluetooth Low Energy only; BLE is low-power but short-range and is not an

internet transport, so a mains-powered station is required as the gateway (section 3).

Compute.
On-device inference runs on a Cortex-M4F with a few hundred kilobytes of RAM and no operating-system heap budget to spare, so models are int8 and arenas are statically sized (section 7).

Connectivity.
The station is provisioned with credentials but never logs in as a user; it authenticates to the cloud with a single revocable token (section 6).

Hosting.
The dashboard is hosted on a managed Python platform (Streamlit Community Cloud), which does not expose server-side cookies, shaping how the session is read (section 10).

Section 14 revisits these requirements and constraints and reports the extent to which each is currently met.

3 System architecture and data flow

This section is the map for the rest of the document. It introduces the four tiers and their responsibilities, the two operating modes, and the four end-to-end data flows that together cover everything the system does: live monitoring, training capture, device onboarding, and model delivery.

3.1 Topology

Meowtion is a four-tier system, fig. 2: the **collar** on the cat, the **station** in the home, the **cloud** backend, and the owner's **dashboard**. The defining constraint is that the collar is BLE-only and battery-powered, with no route to the internet of its own. The mains-powered station bridges that gap, giving the collar a cheap, always-on Bluetooth peer and a Wi-Fi path to the cloud. Everything above the collar is conventional web infrastructure; everything at the collar is shaped by power and radio budget.

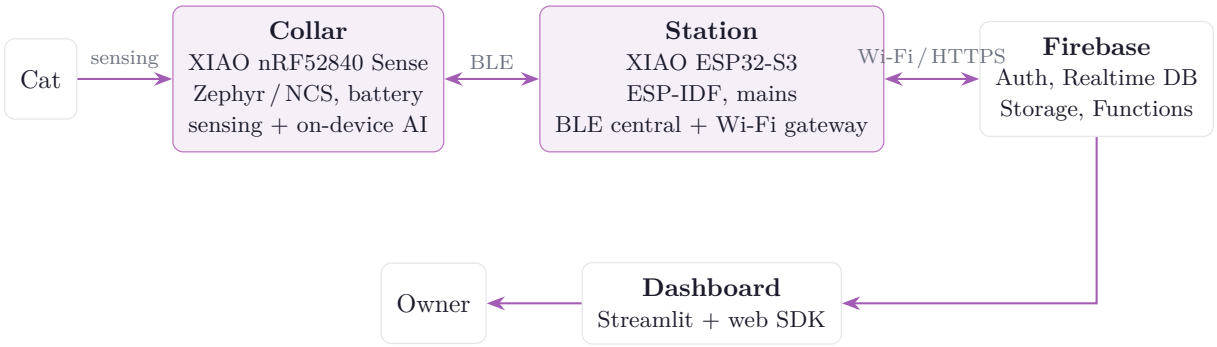


Figure 2: End-to-end topology. The collar is BLE-only, so the always-on station is its gateway to the cloud. The dashboard is a separate client of the same backend.

3.2 Component responsibilities

Table 3 summarises what each tier is responsible for.

Table 3: What each tier is responsible for.

Tier	Responsibility
Collar	Sense (IMU + microphone), classify behaviour on-device, advertise the result as BLE telemetry, and, when a station subscribes, stream paired audio + IMU clips for training and accept models over the air. Holds no credentials.
Station	Scan for and relay registered collars to the cloud; capture and upload training clips; fetch trained models from Storage and push them to the collar over BLE. Authenticates to the cloud with a per-device token, not a login.
Cloud	Firebase: authenticate owners, store the live data and history (Realtime Database), store training clips and trained models (Storage), and run server-side logic (Cloud Functions for training and authenticated clip upload). Enforces per-owner isolation.
Dashboard	The owner-facing web app: a live current-state view and longitudinal health analytics, plus a static front door for sign-in and device registration and a dev console for data capture and training.

3.3 Operating modes

A station runs in one of two modes at a time, selected per station from the dashboard. The mode governs what crosses the BLE link and what the station does with it.

Production mode
The everyday monitoring mode and privacy-preserving default. The collar classifies on-device and advertises only the result; the station reads it and relays it to the cloud. No audio or raw motion leaves the collar.

Training mode
The data-collection mode for teaching a new behaviour. While a registered collar is in range, the station subscribes to the collar's audio service; the collar streams paired audio + IMU clips, which the station slices and uploads and the owner labels and trims in the dev console. It is used deliberately and temporarily, not continuously.

3.4 End-to-end data flows

Four flows fully describe the system's behaviour. Each is given below as a concrete sequence of steps across the tiers; the wire formats are specified in section 12 and the security rules in section 13.

Flow 1 — Live monitoring (production). The steady state of a deployed system.

1. The collar samples its IMU continuously and runs the on-device cascade (section 7); when the motion result is ambiguous it briefly samples the microphone and runs the audio model to confirm.
2. The collar encodes the outcome (class index, confidence, step count, battery) into an 8-byte BLE telemetry advertisement (section 12). Its BLE address identifies the collar, so the payload carries no identifier.
3. The station, scanning continuously, hears the advertisement from a *registered* collar, timestamps it, maps the class index to the owner's label set, and writes the current state plus an event record to the Realtime Database over authenticated HTTPS.

4. The dashboard, reading the same database, refreshes the live card within seconds and folds the new event into the history and health analytics (section 10).

Figure 3: Flow 1, live monitoring. The collar advertises an 8-byte telemetry packet; the station maps the class index to a label and writes the live state and events to the database, which the dashboard reads.

Flow 2 — Training capture. How labelled data is collected to teach a behaviour.

1. In the dev console the owner puts a station into training mode and selects the action being recorded (the "live label").
2. The station connects to the in-range collar and subscribes to its audio characteristic. The collar streams continuously in 100 ms frames, each a header followed by an audio payload and a time-aligned IMU payload (section 12).
3. The station concatenates frames, slices them into fixed-length clips, and POSTs each clip's bytes to the `upload_clip` Cloud Function with its device token. The function verifies the token's owner and writes the clip to that owner's Storage area; the station records the clip's metadata (label, paths, IMU presence) in the database.
4. The owner reviews clips in the dev console: trims the waveform, confirms or changes the label, and discards bad takes. The labelled clips are the training set for Flow 4.

Flow 3 — Device onboarding. How a station and collar are registered to an owner, performed once per device.

1. The owner signs in on the static front door and connects the station over USB using the browser's Web Serial API. The page mints a random per-device *token*, records it under the owner's account (`deviceTokens/<token>/owner`) and as a station device entry, and hands the station its Wi-Fi credentials, the owner id, and the token over the serial link. The station stores these in non-volatile storage; the token is its sole credential.
2. The station comes online, syncs time, and begins scanning, publishing any unregistered collars it hears to its `seen` list.
3. The owner registers a heard collar with one tap in the dashboard. The collar then appears as a device on the account and is relayed by the station. Deleting the token revokes the device.

Flow 4 — Model training and over-the-air delivery. How labelled clips become a model running on the collar.

1. The owner triggers training from the dev console. The `train` Cloud Function reads the account's labelled clips, derives the class set from the labels present, trains an int8 IMU model and an int8 audio model, uploads them to Storage under `models/<uid>/`, and bumps the account's model version (section 8).
2. The station learns a new version is available (mirrored into its own device config), downloads each model from Storage, and, over a brief dedicated BLE connection, streams it to the collar with a CRC-checked begin/data/end protocol (section 12).

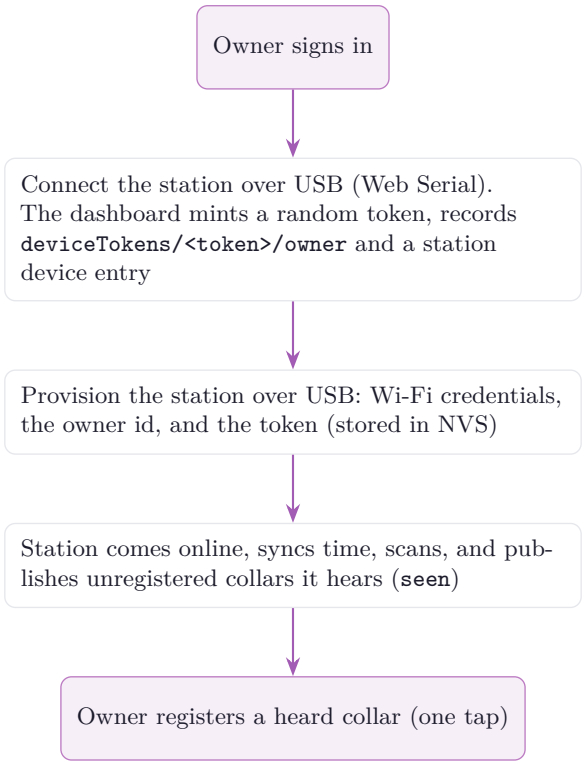


Figure 4: Flow 3, connecting devices. A station is paired over USB and provisioned with a scoped token; once online it surfaces nearby collars, which the owner registers with one tap.

3. The collar stages the model into its dedicated flash partition, verifies the CRC, loads it, and switches from advertising simulated state to advertising real classifier output. The path requires no firmware reflash.

3.5 Multi-tenant data model

Meowtion is multi-tenant. Each owner creates an account and registers their own station(s) and collar(s); all of an owner's data lives under `users/<uid>/` in the Realtime Database, and the database and storage rules scope every read and write to the owning account (sections 9 and 13). The backend enforces isolation, not the client. Battery devices never hold a login: a station authenticates with its scoped, revocable token, and a collar holds nothing at all (the station relays it). A single public, read-only demo account, designated in server-side config, lets anyone explore a populated dashboard without signing up.

3.6 Technology summary

Table 4 lists the technology used at each tier.

Table 4: Technology at each tier.

Tier	Stack
Collar	Seeed XIAO nRF52840 Sense; Zephyr / nRF Connect SDK (v3.3.1); TensorFlow Lite Micro with CMSIS-NN kernels
Station	Seeed XIAO ESP32-S3; ESP-IDF; NimBLE (BLE central) + Wi-Fi station
Cloud	Firebase: Authentication, Realtime Database, Cloud Storage, Python Cloud Functions (2nd gen)
Dashboard	Streamlit (Python) + Firebase web SDK (JavaScript); hosted on Streamlit Community Cloud
Training	TensorFlow / Keras, executed server-side in a Cloud Function; int8 post-training quantisation to TensorFlow Lite

PART II

The device

4 Hardware

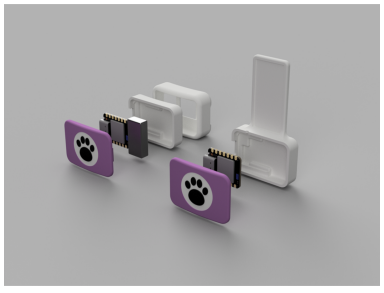
The build is two devices: the collar worn by the cat and the station plugged in at home. There is no custom PCB. Each device is a Seeed Studio XIAO module in a 3D-printed shell, chosen for the small footprint, integrated radios, hand-solderable castellated pads, and, on the collar, on-module sensors. The full parts list is in section A; print files and assembly steps are in `hardware/README.md`. This section summarises the design and its reasoning.

4.1 Collar

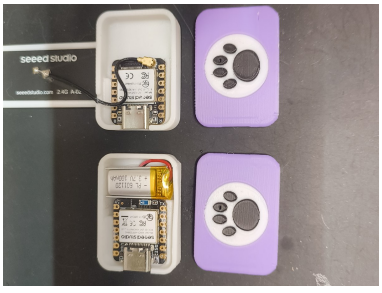
The collar is built around the Seeed XIAO nRF52840 Sense. The nRF52840 provides a Bluetooth Low Energy radio and enough flash and RAM to run quantised neural-network inference on its Cortex-M4F. Critically for Meowtion, the *Sense* variant carries both sensors the system needs on the module itself, a six-axis IMU (LSM6DS3TR-C) and a PDM MEMS microphone, so the collar needs no external sensor breakout and stays small and light.

The collar runs on its own 1S LiPo cell (3.7 V, 100 mAh, with a built-in protection circuit) soldered to the XIAO's BAT pads and charged through the module's onboard divider. The build is completed by a 20 mm hydrophobic PTFE filter membrane over the mic hole, PETG front and back shells, a flexible TPU skin, and a 10 mm quick-release safety strap. The full parts list is in section A.

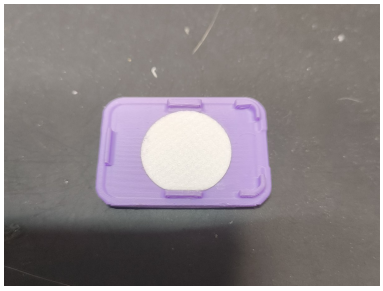
The IMU supplies the motion stream that is always classified; the microphone supplies the short audio window used to disambiguate behaviours that look alike by motion but differ by sound (section 7). The microphone port sits behind the PTFE membrane in the front shell; correct membrane placement matters for audio quality and water resistance and is documented in the hardware guide.



(a) Exploded view.



(b) Assembled internals.



(c) PTFE membrane over the mic hole.

Figure 5: Collar and station hardware. Full parts, print files, and assembly steps are in `hardware/README.md`.

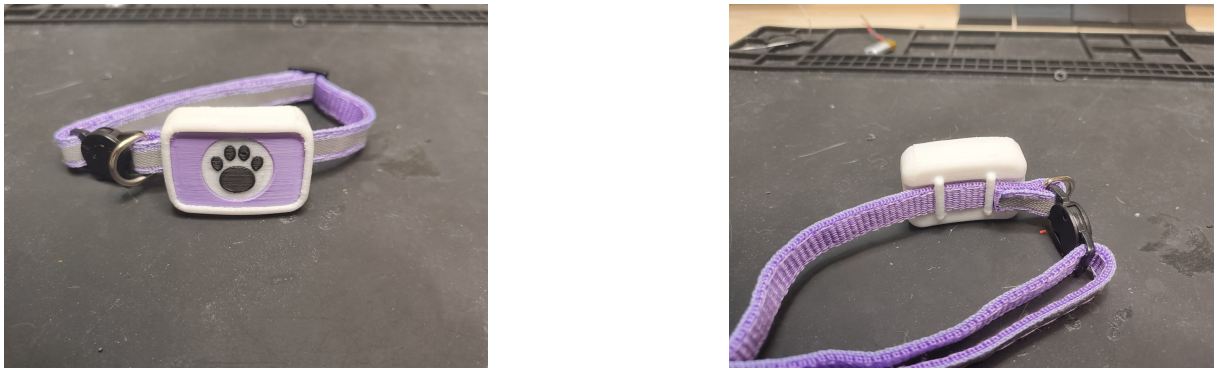
4.2 Station

The station is a Seeed XIAO ESP32-S3. The ESP32-S3 has both Wi-Fi and BLE, so a single module acts as the BLE central that listens to the collar and the Wi-Fi client that reaches the cloud. Mains-powered over USB-C, it scans and stays online continuously without the wearable's power constraints. It carries no sensors of its own; its job is to hear the collar, capture training

clips when asked, relay results to the cloud, and push models to the collar. Its parts are listed in section A.

4.3 Enclosure and assembly

The front shell is shared between the collar and the station; the back shells and the collar's flexible TPU skin are per-device. The collar shells are PETG for rigidity; the station back shell is any rigid filament. The collar is assembled by seating the board and cell in the back shell, fitting the PTFE membrane over the mic hole, bonding the shell halves with water-resistant glue, and slipping the sealed unit into the TPU skin on a quick-release strap. The station is seated, glued closed, and plugged in by the bowl. The hardware designs are released under CERN-OHL-S v2 (strongly reciprocal open hardware).



(a) Front, on the strap. (b) Back, on the strap.

Figure 6: The finished collar on a quick-release strap.

4.4 Rationale for the split

The collar must be small, light, and run for a useful time on a small cell, which rules out Wi-Fi on the wearable: its power draw is incompatible with a collar-sized battery. BLE is low-power but short-range and is not an internet transport. The fixed, mains-powered station resolves both problems, giving the collar a cheap always-on BLE peer and a Wi-Fi path to the cloud. The split also keeps the privacy boundary clean: the heavy and sensitive processing (including all audio) stays on the collar, and only a classification result crosses to the station.

5 Collar firmware

The collar runs on Zephyr [5], a real-time operating system for microcontrollers, via the nRF Connect SDK (NCS v3.3.1), targeting board `xiao_ble/nrf52840/sense`. It is a connectable BLE peripheral that senses on the cat, advertises a compact telemetry packet, and streams paired audio + IMU clips for training once a station connects and subscribes. It never touches Wi-Fi or the internet, so it stays small, low-power, and battery-only. `build.ps1` drives the build: it locates the local NCS toolchain, runs `west build -b xiao_ble`, and copies out the UF2.

5.1 Responsibilities

The firmware has four jobs, driven from the main loop and BLE callbacks:

1. Run the rules-based activity gate: while the cat is active, sample the IMU and run the classifier cascade; on sustained stillness drop to low-power rest until motion returns.

2. Advertise the result as an 8-byte BLE telemetry packet for the station to read (section 12).
3. In training mode, stream paired audio + IMU frames to a subscribed station for capture.
4. Receive new models over the air and swap them in safely.

5.2 Source layout

Table 5 lists the collar firmware modules and what each is responsible for.

Table 5: Collar firmware modules (`firmware/collar/src/`).

File	Responsibility
`main.c`	Boot and the telemetry loop; orchestration only
`ble.c`	Advertising, identity, the audio GATT service, notifications
`audio.c`	PDM microphone capture, decimated to 8 kHz μ-law
`audio_codec.h`	μ-law codec and the model-input audio representation
`streaming.c`	Decoupled reader/sender threads that assemble and send clip frames
`imu.c`	LSM6DS3TR-C continuous sampler
`battery.c`	1S LiPo charge state via the onboard divider
`classifier.c`	The confidence-gated cascade policy (IMU first, audio confirms)
`activity.c`	The rules-based activity gate (cascade tier 0): still-timer that drives low-power rest
`tflm_classifier.cpp`	TensorFlow Lite Micro backend for the cascade
`model_loader.h`	Runtime model-install API (`clf_set_imu_model` / `clf_set_audio_model`)
`production.c`	Rolling IMU window; runs inference and writes real telemetry
`ota.c`	OTA receive state machine; stages models to flash and defers the swap

The classifier is split deliberately. `classifier.c` defines the cascade policy in plain C with *weak* hooks for model-presence queries and inference calls; `tflm_classifier.cpp` provides the *strong* `extern "C"` definitions that override them when the TensorFlow Lite Micro backend is compiled in. This separates the policy from the heavy C++ inference, so the firmware links and runs even with the backend or any model absent.

5.3 Streaming and threading

Capturing training data is the collar's most timing-sensitive path: PDM audio arrives steadily and must be paired with IMU samples and pushed over BLE without dropping frames. The reader and sender are therefore decoupled (`streaming.c`): one side assembles 100 ms frames from the audio and IMU sources, the other sends them over the audio characteristic, so a momentarily slow radio cannot stall capture. The collar negotiates a large ATT MTU (247 bytes) and uses the 2 Mbit PHY for throughput.

5.4 Activity gating and low-power rest

The collar's power budget (NFR-1) is set by how much it senses, not by how fast it can classify, and a cat is at rest for most of the day. A rules-based activity gate (`activity.c`), the first tier of the cascade (section 7), turns that idleness into battery life. Each production cycle it takes a motion-energy proxy, the peak deviation of acceleration from 1 g over the cycle's samples, and runs a still-timer: once motion stays below a small threshold for a hold-off window (about a minute), the collar enters *low-power rest*.

In rest, the 104 Hz sampling thread and all inference stop, the IMU drops to a low-power watch rate (about 12.5 Hz), and advertising slows to roughly a one-second interval; the main thread polls the accelerometer slowly and the SoC idles between polls. A motion event above a wake threshold ends rest: the IMU and advertising return to their active rates and the cascade resumes. Because the station logs an episode whenever the collar's reported state changes, the collar advertises a dedicated `REST` state byte while resting, so the entire dormant span is recorded as a single *rest* event covering exactly the period of low-to-no motion (sections 9 and 12). The thresholds and hold-off are constants in `activity.h`, tuned against the measured power budget (section 14).

5.5 Model storage and flash layout

Models are not linked into the firmware image. They live in a dedicated flash partition so a new model can be written over the air without reflashing. The partition map is fixed in `pm_static.yml` to stop the partition manager from regrowing the application region over the model storage:

Table 6: Collar flash partitions (`pm_static.yml`).

Partition	Address / size	Contents
`app`	`0x27000` / `0x85000`	Application code (fixed size)
`models_storage`	`0xAC000` / `0x40000` (256 KiB)	Header plus the IMU (slot 0) and audio (slot 1) model blobs
`settings_storage`	`0xEC000` / `0x08000`	NVS settings
`uf2_partition`	`0xF4000` / `0x0C000`	UF2 bootloader (unchanged)

5.6 Build configuration

On-device inference requires pulling TensorFlow Lite Micro into the NCS workspace (an optional, normally-excluded west module) and turning on C++17 and the TFLM and CMSIS-NN options. Two stability fixes are baked into the configuration: TFLM's `AllocateTensors()` and `Invoke()` overflow Zephyr's small default stack, so the main stack is raised; and the model storage uses `stream_flash` to lazily erase pages during an OTA write.

```
CONFIG_CPP=y
CONFIG_STD_CPP17=y
CONFIG_TENSORFLOW_LITE_MICRO=y
CONFIG_TENSORFLOW_LITE_MICRO_CMSIS_NN_KERNELS=y
CONFIG_MAIN_STACK_SIZE=16384      /* AllocateTensors()/Invoke() overflow the ~2 KiB
     default */
CONFIG_STREAM_FLASH_ERASE=y       /* lazily erase model pages during OTA */
CONFIG_BT_L2CAP_TX_MTU=247        /* large MTU for audio streaming + OTA chunks */
CONFIG_BT_CTLR_PHY_2M=y           /* 2 Mbit PHY for throughput */
```

Listing 1: Inference-related `prj.conf` settings (collar).

The OTA receive path stages bytes to flash and defers the verify-and-swap onto the main thread rather than committing a model from the BLE-RX callback (section 12).

5.7 Development console and flashing

The XIAO nRF52840 has no debug COM port, so flashing is always a UF2 drive-copy: double-tap RESET to mount the bootloader drive, then re-run the build script to copy the UF2 across. For development the firmware brings up a USB CDC ACM console on the *legacy* USB device

stack; the newer NEXT stack's CDC console drops output on this board. The console prints a `MEOW> collar id=...` banner and a periodic `state= steps= batt=` line; the dashboard reads the `id=` banner over Web Serial to register the collar.

5.8 Telemetry output

In production the telemetry packet carries the cascade's live result: class and confidence from the on-device classifier on the rolling IMU window, with the audio stage confirming uncertain results (section 7). It also reports an IMU-derived step count and the measured battery level. While the activity gate holds the collar in low-power rest it advertises a dedicated `REST` state instead, which the station records as a rest episode (section 12). A freshly flashed collar advertises `UNKNOWN` until its first model arrives over the air, so the firmware is always safe to run.

6 Station firmware

The station runs on ESP-IDF [6], Espressif's SDK for the ESP32, on the XIAO ESP32-S3. It is a BLE central that hears registered collars and a Wi-Fi gateway that relays them to the owner's Firebase account. It also captures short audio + IMU training clips and uploads them, and it delivers trained models to the collar over BLE. It has no Firebase login: Wi-Fi credentials, the owner id, and a per-device token are provisioned over USB serial and stored in non-volatile storage (NVS), and the database authorises the station because that token is registered to the owner.

6.1 Source layout

Table 7 lists the station firmware modules and what each is responsible for.

Table 7: Station firmware modules (`firmware/station/main/`).

File	Responsibility
`main.c`	Boot and the roughly one-second orchestration loop
`config.c`	Provisioned credentials, NVS storage, USB-serial provisioning
`board.c`	Power-source detection (USB vs battery)
`wifi.c`	Wi-Fi station bring-up and time synchronisation
`cloud.c`	The Firebase Realtime Database / HTTP layer (the only HTTP in the firmware)
`weather.c`	Local weather from the Open-Meteo feed
`ble.c`	The whole NimBLE collar subsystem (scan, relay, audio capture) behind `ble_service()`
`ota.c`	OTA model delivery: pushes trained `.tflite` models to the collar over BLE

6.2 Provisioning

Provisioning is a single JSON line sent over USB serial. The dashboard's "Connect a device" form does this automatically over Web Serial as part of the onboarding flow (section 3); the line is plain text and can be sent by hand for debugging:

```
1 {"ssid":"..","pass":"..","owner":"<uid>","token":"<random>","name":"..","lat
      ":50.8,"lon":-0.1}
```

Listing 2: Provisioning line (USB serial).

The station stores these in NVS and can be re-provisioned from the dashboard at any time, or wiped with `idf.py erase-flash`. The token is the station's only credential; deleting it from the owner's account revokes the device immediately. Repository secrets such as `secrets.h` are excluded by `.gitignore`, so no live credential is tracked.

6.3 Relay and clip upload

In production mode the station scans continuously, hears the telemetry advertisements of registered collars, maps the compact result (mode, class index, confidence) onto the cat's record, and writes both a live `current` state and discrete activity events as episodes begin and end (`cloud.c`). Events carry the model version and class index rather than a resolved name, so the dashboard can map them to labels even as the label set changes (section 9). In training mode it connects to an in-range collar, captures clips, and POSTs each clip's bytes to the `upload_clip` Cloud Function with its device token; the function verifies the token's owner and writes the file to that owner's Storage area, since clients cannot write Storage directly. A manual capture path ignores the signal-strength gate, to collect behaviours such as purring that happen while the cat rests rather than at the bowl.

6.4 Over-the-air model delivery

The station delivers trained models, since the collar is BLE-only and never touches Storage. The wire protocol is fixed in the shared header `firmware/common/meow_ota.h`: the collar is the GATT server that stages a `.tflite` into flash, the station is the client that pushes it (section 12).

Version gate.
The `train` function bumps `users/<uid>/models/version` and mirrors it into this station's own config as `modelVer` (the station may read only its own `users/<uid>/devices/<token>/` subtree, not `users/<uid>/models`). The station keeps its last-pushed version in NVS; a push is due when the config version is higher.

Download.
For each slot it fetches the model from Storage at the public-read path `models/<uid>/imu_model.tflite` (IMU slot) or `models/<uid>/audio_model.tflite` (audio slot). A 404 means that slot has no model yet and is skipped.

Push.
It computes a CRC-32/IEEE over the bytes and pushes the model over a brief dedicated BLE connection using the BEGIN/DATA/END/CRC sequence (section 12). On the collar's `OK` it records the new version in NVS; on any error it leaves NVS unchanged and retries later.

A push runs only when a registered collar is in range and audio capture is idle, so model delivery never contends with the capture or relay path for the radio.

PART III

On-device intelligence

7 On-device AI

All classification happens on the collar in real time, using TensorFlow Lite Micro [7] with the CMSIS-NN kernels [8] for the nRF52840's Cortex-M4F. The models are tiny int8 networks in the spirit of TinyML [9]: small enough to fit a microcontroller's RAM and run within the collar's power budget, yet accurate enough to separate everyday behaviours.

7.1 Three-tier cascade

Inference cost rises sharply across the cascade, so Meowtion gates each tier behind a cheaper one and pays for the next only when the cheaper tier cannot answer. Three tiers run in front of every reported behaviour:

1. **Rules-based activity gate** (no model). A cheap motion-energy rule on the raw IMU stream decides *active* vs *resting*. Most of a cat's day is rest, and classifying stillness 50 times a second drains the cell for no information, so sustained low motion drops the collar into low-power rest: the 104 Hz sampling and inference stop, the IMU falls back to a low-power motion watch, and the radio slows. Motion ends rest, and the whole dormant span is reported as a single *rest* episode (section 5). This gate is the dominant battery lever (section 14).
2. **IMU model** (model-based). While the cat is active, the int8 motion model runs on every window. It is cheap and carries most of the classification load.
3. **Audio model** (model-based). When the IMU model is *unsure* (its top class falls below a confidence threshold of 0.75), the short audio model is invoked to confirm. Audio is therefore a deployed disambiguation signal, not merely a training aid.

The audio tier earns its keep on one hard case: eating and drinking look almost identical to a motion sensor (head down at a bowl, similar gross motion) but sound completely different. The cascade keeps average power low at both ends , the rules-based gate suppresses all sensing and inference while the cat rests, and the audio path runs only to break a tie , while still recovering accuracy that motion alone cannot provide. The policy lives in plain C, independent of the inference backend: the rules-based gate in `activity.c`, and the model-tier gating in `classifier.c`. Figure 7 shows the decision flow.

7.2 Matching training and inference

The on-device input transform must mirror the training transform exactly; this is the single most important correctness property. A model trained on one representation and fed another at inference produces confident nonsense. Meowtion enforces the match on both paths:

- **Motion.** The rolling window is normalised per channel (subtract each channel's mean over the window, divide by the global maximum absolute value with a small epsilon) and quantised to int8 using the input tensor's own scale and zero-point.
- **Audio.** Inference audio is decimated and encoded to match the training representation: 8 kHz μ-law, produced by the same conversion used to prepare the training clips (section 12). A mismatch here was an early source of confident misclassification.

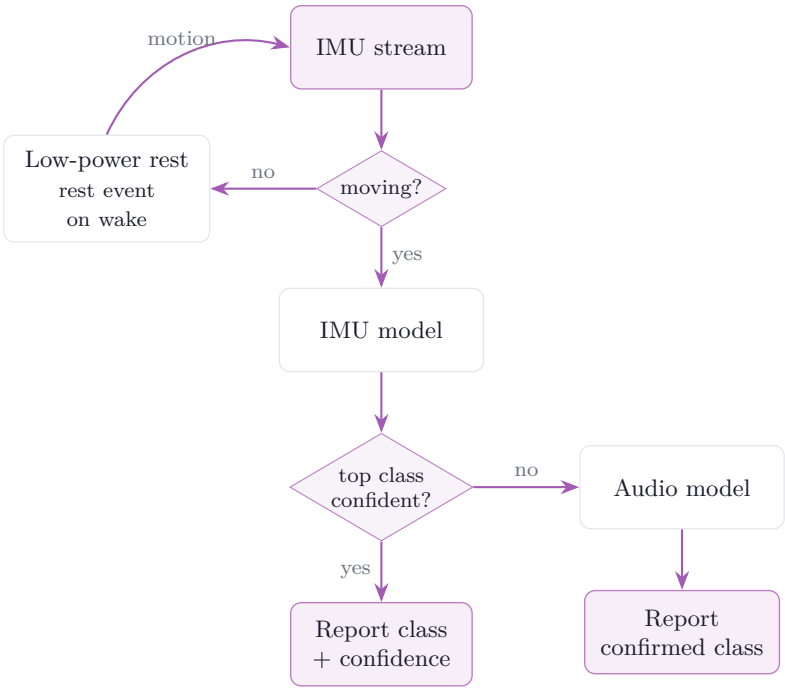


Figure 7: The three-tier cascade. The rules-based activity gate runs first: while the cat is still the collar rests in low power and the dormant span is logged as one rest event; on motion the IMU model runs on every window, and the audio model is invoked only when the IMU model's top class is below the confidence threshold. With no model loaded the model tiers return `UNKNOWN`.

To avoid a large floating-point scratch buffer on a RAM-constrained part, the IMU path quantises directly from the int16 source in a few passes (per-channel mean, then global max-abs, then write int8) rather than materialising a float window (listing 3).

```
// pass 1: per-channel mean over the window
// pass 2: global max-abs of (sample - channel_mean)
// pass 3: int8 = round( x_norm / scale ) + zero_point,
//         where x_norm = (sample - channel_mean) / (maxabs + 1e-6)
// scale and zero_point are read from the input tensor itself.
```

Listing 3: Direct int16-to-int8 quantisation, mirroring training (sketch).

7.3 Model architecture

Each model stage is a small 1-D convolutional network trained in the cloud and exported as an int8 `.tflite` (section 8). The IMU stage runs over a rolling window of motion samples (six channels: accelerometer and gyroscope x/y/z); the audio stage runs over one second of 8 kHz audio (8000 samples).

Both stages are the small 1-D CNNs defined in the training pipeline (section 8). Every convolution uses `same` padding and ReLU; with a stride s the output length is $\lceil L/s \rceil$. The classifier head has N outputs, where N is the number of action classes discovered from the labelled data, so the head and the totals scale with N. The exported int8 model sizes are reported in table 17.

7.4 Runtime models and model-free safety

No model is compiled into the firmware. Each stage loads its `.tflite` at runtime (`clf_set_imu_model` / `clf_set_audio_model`) from the dedicated flash partition, de-

Table 8: Per-stage network architecture. `k` = kernel size, `s` = stride; N is the action-class count (discovered from the data). Parameter counts are for the trained float model; export quantises weights to int8.

Stage	Layer	Output	Params
IMU	Input	104×6	0
	Conv1D 16, k5, s1, ReLU	104×16	496
	MaxPool1D 2	52×16	0
	Conv1D 32, k5, s1, ReLU	52×32	2 592
	MaxPool1D 2	26×32	0
	Conv1D 64, k3, s1, ReLU	26×64	6 208
	GlobalAvgPool1D	64	0
	Dense 32, ReLU (Dropout 0.3)	32	2 080
	Dense N, Softmax	N	$33N$
	Total		$11\,376 + 33N$
Audio	Input	8000×1	0
	Conv1D 8, k9, s4, ReLU	2000×8	80
	Conv1D 16, k9, s4, ReLU	500×16	1 168
	Conv1D 32, k5, s2, ReLU	250×32	2 592
	Conv1D 32, k3, s2, ReLU	125×32	3 104
	GlobalAvgPool1D	32	0
	Dense 32, ReLU (Dropout 0.3)	32	1 056
	Dense N, Softmax	N	$33N$
	Total		$8\,000 + 33N$

livered over the air (sections 5 and 12), so a re-trained model swaps in without reflashing. The engine is also safe with no model present: the presence queries return false and the cascade returns `UNKNOWN` rather than a wrong answer. This is the state of a freshly flashed collar before its first model arrives.

7.5 Memory budget

TensorFlow Lite Micro uses a fixed, statically allocated tensor arena (this firmware uses no heap). The two stages never infer at once , the cascade runs the motion model first and consults audio only when unsure , so rather than reserve an arena each, they share a *single* arena: the collar loads exactly one model into it at a time and unloads that model before arming the other, so only one model's tensors are ever resident. The arena is sized to the larger (audio) stage; the smaller motion model fits inside it. The value in table 9 is a starting budget and must be tuned to the real exported models, since an undersized arena makes `AllocateTensors()` fail and that stage report itself absent.

Table 9: Shared tensor arena (initial budget).

Arena	Size	Notes
Shared (one model resident)	48 KiB	Sized to the audio stage (1-D conv over 8000 points); the motion model loads into the same arena and fits. The two flip in and out on demand, so only one model's tensors occupy RAM at a time

Sharing one arena rather than two is a deliberate choice on the memory-bound collar: it saves the second arena outright. Enabling inference initially pushed RAM use to roughly 94 %; eliminating the float-window scratch buffer (quantising directly from int16) and sharing the

single arena brought it back into the mid-80s, leaving headroom for the BLE stack and the raised main stack (section 5).

8 Training pipeline

Meowtion learns new behaviours through a closed loop the owner drives entirely from the dashboard, with no local toolchain: capture, label, train, deliver, run. The loop turns the collar from a fixed-function sensor into a platform for recognising any behaviour the owner can demonstrate.

8.1 Closed loop

1. **Define behaviours.** The owner lists the actions to recognise (eat, drink, purr, scratch, litter-tray, and so on) in the dev console. This set drives both the clip-label dropdown and the trained model's outputs.
2. **Capture.** With a station in training mode, paired audio and time-aligned motion are recorded while a registered collar is in range and uploaded to Cloud Storage (Flow 2, section 3).
3. **Label.** The owner reviews clips in the dev console, sets each clip's action, trims dead space from the waveform, and discards poor examples.
4. **Train.** The `train` Cloud Function trains the models server-side from the labelled clips and writes the results back: the quantised `.tflite` blobs to Cloud Storage, and the new version and label list to the Realtime Database.
5. **Deliver.** Each station watches a model-version key; when it changes the station downloads the new `.tflite` files and pushes them to the collar over the air via the BEGIN/DATA/END/CRC sequence (section 12).
6. **Run.** In production mode the collar classifies with the new model and the dashboard shows live, named behaviours. Observing those results, the owner captures more clips or adds behaviours and retrains, closing the loop: the collar's own data improves the model it runs (fig. 8).

Figure 9 shows how the training *data* moves through the system during the capture and label steps; fig. 10 (below) shows how the trained *model* moves back out.

8.2 The trainer

Training runs in the `train` Python Cloud Function, currently gated to developer accounts to keep cloud costs down during development rather than as an inherent limit (section 9). It reads the account's labelled clips (metadata from the Realtime Database, audio and motion bytes from Cloud Storage) and derives the class set directly from the labels present, so the network's outputs follow whatever behaviours the owner defined. It then trains the two cascade stages independently:

- a small 1-D convolutional network over the motion windows (six IMU channels), and
- a larger 1-D convolutional network over the 8 kHz audio windows.

Each input is windowed with overlap and normalised exactly as the collar normalises it at inference, so the on-device and training transforms match (section 7). The trained Keras models are converted to TensorFlow Lite with full int8 post-training quantisation using a representative dataset, so the exported `.tflite` weights and activations are int8 and the input and output tensors carry the scale and zero-point the collar uses to quantise its own inputs. Figure 10 shows the whole path, from the training trigger to the model running on the collar.

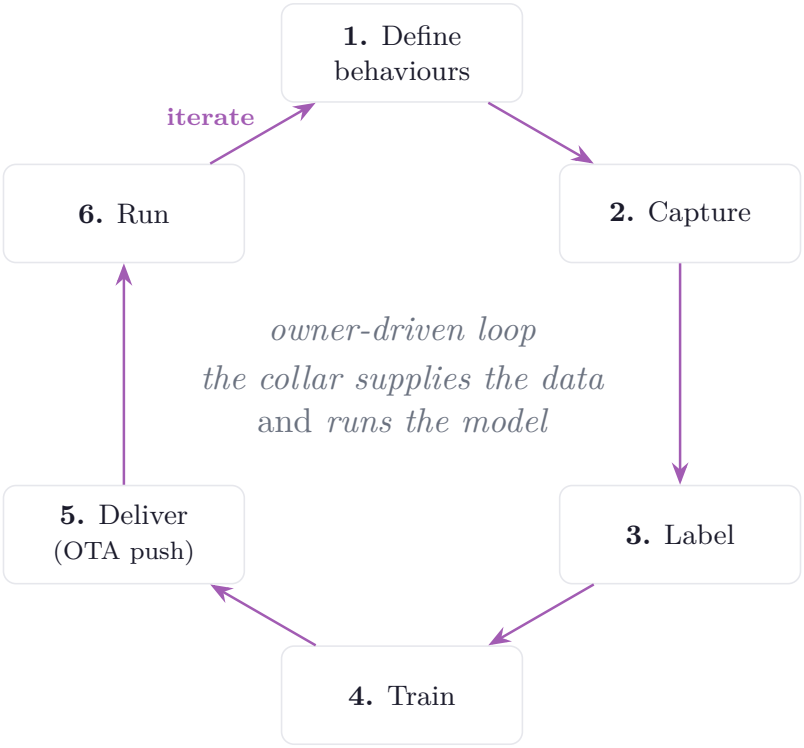


Figure 8: The training pipeline closes into a loop. The same collar that captures the labelled data (steps 2–3) also runs the model trained from it (step 6); observing the live results, the owner refines or adds behaviours and retrains, so each pass improves the next. The loop is owner-driven (human-in-the-loop), not autonomous.

8.3 Label-agnostic by design

The trainer is not hard-coded to a fixed set of actions; it learns whatever label set the owner defines. This makes Meowtion a platform rather than a fixed-function gadget: the same pipeline that recognises eating and drinking can be pointed at any distinguishable behaviour the owner can capture and label. The class set flows end to end as data, from the dashboard's action list, through the trainer, to the index-based on-collar classifier, with names attached only at the edges.

8.4 Delivery contract

Training outputs are written to predictable locations so the station finds them without coordination: the model blobs land in Cloud Storage as `models/<uid>/imu_model.tflite` and `models/<uid>/audio_model.tflite`, the new version is mirrored into each of the owner's stations' configuration so their version gate fires, and the label list is stored in `users/<uid>/models` so the dashboard can map a class index back to a name across label-set changes (section 9).

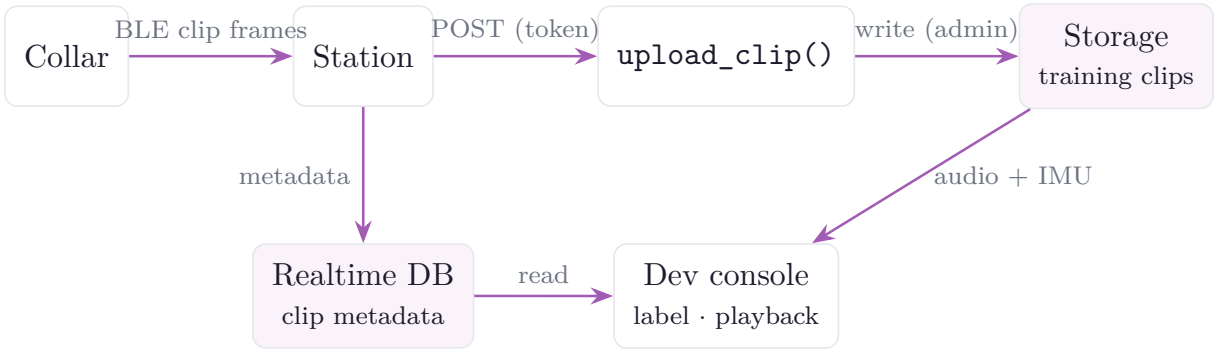


Figure 9: Training-data transport. The collar streams paired audio + IMU clip frames to the station, which slices them into clips and uploads the bytes through `upload_clip` to Storage and the clip metadata to the database; the dev console reads both to play back, label, and train on them.

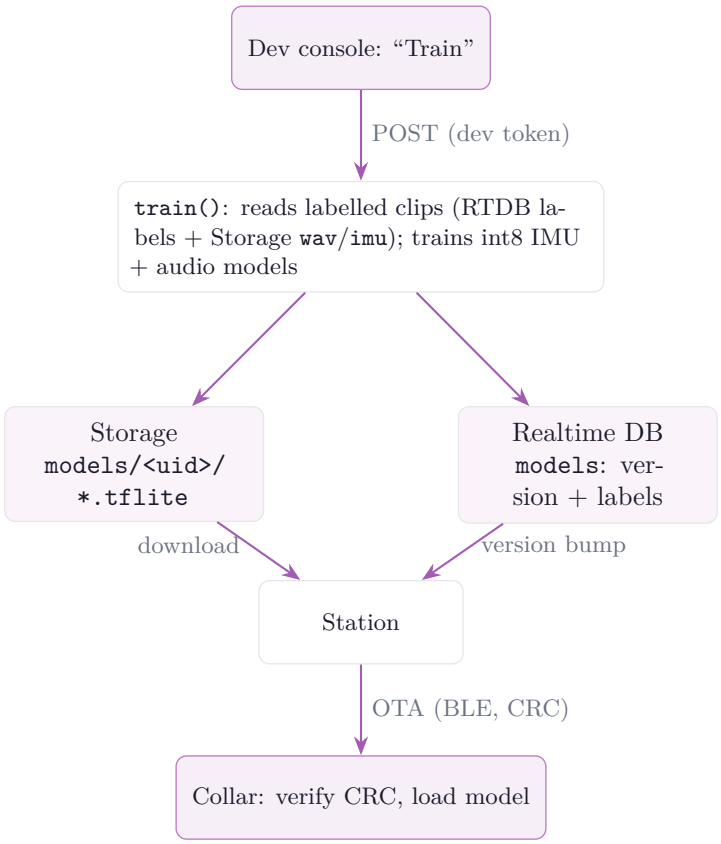


Figure 10: Training and over-the-air model delivery. `train()` reads the labelled clips, writes the quantised models to Storage and the new version and labels to the database, and the station detects the version bump, downloads the models, and pushes them to the collar.

PART IV

Cloud and application

9 Cloud backend

The backend is Firebase [10], Google's hosted backend platform (sign-in, database, file storage, and serverless functions). It provides owner authentication, a Realtime Database (RTDB) for live state and history, Cloud Storage for training clips and model blobs, and Python Cloud Functions (2nd generation) for server-side logic. There is no custom server to operate. The whole backend lives in `app/firebase/` and deploys with `firebase deploy` from that folder.

9.1 Authentication

Owners sign in with Firebase Authentication (email/password or Google). The collar and station do *not* authenticate as users: a station carries a scoped per-device token, and a collar carries nothing (the station relays it). The credential surface on battery devices is thus a single revocable token (section 13).

9.2 Backend data flow

Figure 11 shows how the tiers exchange data in normal operation. The dashboard reads the RTDB, both a small per-cat live read and a cached history read (section 10). The station writes telemetry and events to the RTDB and uploads training clips to Cloud Storage through the `upload_clip` function. The station also relays the collar and delivers models to it over BLE. Clients never write Cloud Storage directly (section 13). The training path that produces those models appears separately in fig. 10.

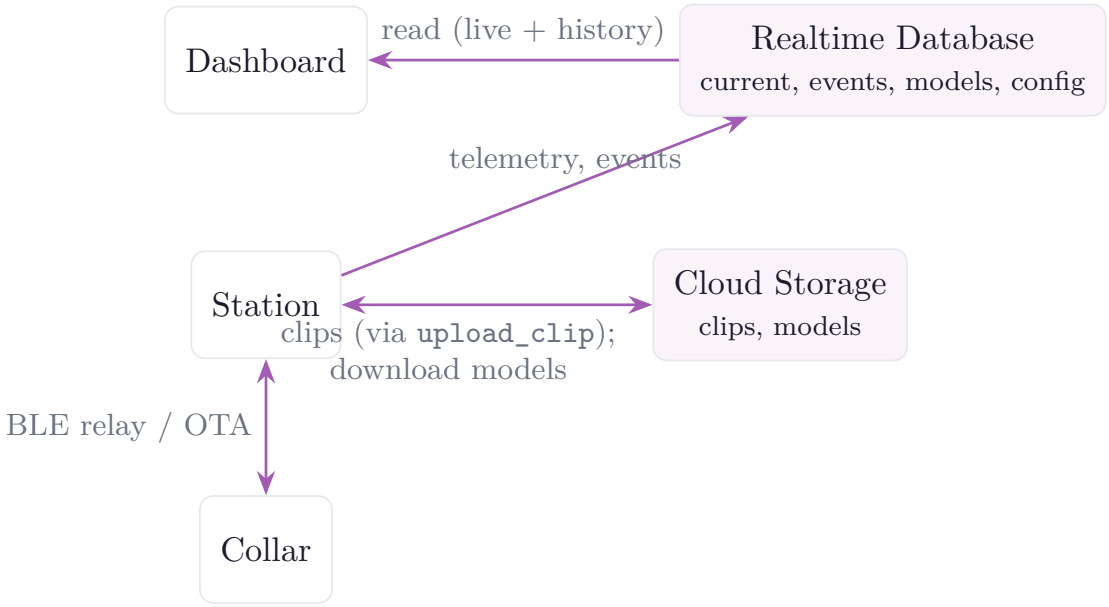


Figure 11: Backend data flow in normal operation. The dashboard reads the database; the station writes live state and events and uploads clips; the collar is relayed and updated through the station.

9.3 Realtime Database schema

State and history live in the RTDB, keyed by owner so the security rules scope every read and write to the owning account. Table 10 lists the main paths.

Table 10: Realtime Database paths.

Path	Contents
`users/<uid>/devices/<token>`	A registered station (`type:station`) with its relayed `cats/<catId>` subtree, or a registered collar entry
`.../cats/<catId>/current`	The cat's live state (version, class, confidence, steps, battery) plus the station-measured signal (`rssi`) and `proximity` , the single source for the dev console's presence view
`.../cats/<catId>/events`	Discrete activity events (carry model version + class index)
`.../devices/<station>/weather`	Current and recent local weather for the station's location
`users/<uid>/models`	Current model version, status, and the label list
`users/<uid>/actions`	The behaviours to recognise (managed from the dev console)
`deviceTokens/<token>/owner`	Maps a device token to its owning account
`config/demoOwner`	UID whose data backs the public read-only demo
`config/devAccounts/<uid>`	Accounts allowed to use the dev/training tools

Events store the model version and class index rather than a pre-resolved name. The dashboard maps the index to a label using that version's label list, so historical events stay correct as the owner changes their behaviour set.

9.4 Cloud Storage

Cloud Storage holds the training clips (audio plus aligned motion) and the trained model blobs in an owner-scoped layout: clips at `training/<owner>/<collar>/<ts>.{wav,imu}` and models at `models/<uid>/{imu,audio}_model.tflite`. Training clips are private to the owner and *not* world-readable, even in the demo, because home audio must never be public. Model blobs are public-read so the token-only station can fetch one to push to the collar, and are written only by the trainer.

9.5 Cloud Functions

Two request functions handle the owner-facing server-side logic, and the demo simulator (below) is a third, scheduled function. All run on the function's ambient service account, so no service-account key is in the repository.

`train`
: Server-side model training, `POST` with a developer account's Firebase ID token and gated additionally by `config/devAccounts/<uid>`. It reads the account's labelled clips, trains the int8 IMU and audio models (section 8), uploads them to `models/<uid>/`, updates `users/<uid>/models`, and mirrors the new version into each of the owner's station configs to trigger delivery.

`upload_clip`
: Authenticated clip upload for the token-only station. The station has no login, so it cannot satisfy the Cloud Storage rules directly. It POSTs the clip bytes here with its device token; the function verifies `deviceTokens/<token>/owner` and writes the file as administrator into

that owner's area. The rules can then deny all direct client writes while still admitting an authorised station.

9.6 Demo data simulator

So the public demo always presents a populated, believable dashboard, a scheduled Cloud Function maintains a fully simulated companion collar ("Purrminator") on the demo account. It is standalone: no station, always online. On first run it backfills roughly six months of realistic activity history, weighted by time of day to match an average cat (crepuscular, busy around dawn and dusk, resting overnight), with matching daily weather from a free historical feed. Thereafter it wakes on a short schedule and commits each activity as an event when that action ends, so new events arrive at varying intervals. The simulator writes only the demo account's own data and never touches a real collar.

9.7 Weather context

Local weather lets the dashboard read a habit change against its environment, since a warm spell that raises drinking and suppresses appetite should not be mistaken for illness. The station polls a keyless forecast feed (Open-Meteo) for its provisioned location and writes current and recent weather under its device record. The demo simulator backfills weather from the same provider's historical archive alongside the simulated activity.

9.8 Configuration trust boundary

The `config` subtree is never client-writable and is maintained from the Firebase console. Reads are scoped to just what a client needs: `config/demoOwner` is public (it backs the demo) and a signed-in user may read only their own `config/devAccounts/<uid>` flag, so the developer list cannot be enumerated and a client still cannot grant itself privileges.

10 Dashboard

The owner-facing application is a dashboard built with Streamlit [11] (a Python framework for data web apps), backed by a small set of static pages for the parts that need the Firebase web SDK directly (sign-in, account and device management, the developer/training console, and the privacy policy). It is hosted on Streamlit Community Cloud and deploys automatically from the repository's main branch.

10.1 Hybrid front end

Device registration needs the browser's Web Serial API, which cannot run in Streamlit's Python or its sandboxed component iframes, and authentication is easiest with the Firebase web SDK in the browser. The data views and charts, by contrast, are easiest in Streamlit's Python. Meowtion therefore serves both halves from the same origin so they share a login: the static sign-in page writes the Firebase ID token into a same-origin cookie, and the Python dashboard reads that cookie to identify the owner.

10.2 Navigation

The product is built around a single hub, the static front door. From it the owner signs in to the dashboard, opens the developer console (if their account is enabled for it), or enters the read-only demo; logging out returns there. Figure 12 shows the navigation. A shared visual style, a white canvas with lavender accents and collapsible cards, runs across every surface.

Table 11: Dashboard surfaces.

Surface	Tech	Role
`app.py`	Streamlit/Python	Live cat state, activity history, health analytics
`account.html`	JavaScript	Sign-in, account management, device pairing over Web Serial
`dev.html`	JavaScript	Developer/training console: capture, label, train, mode
`privacy.html`	JavaScript	Privacy policy

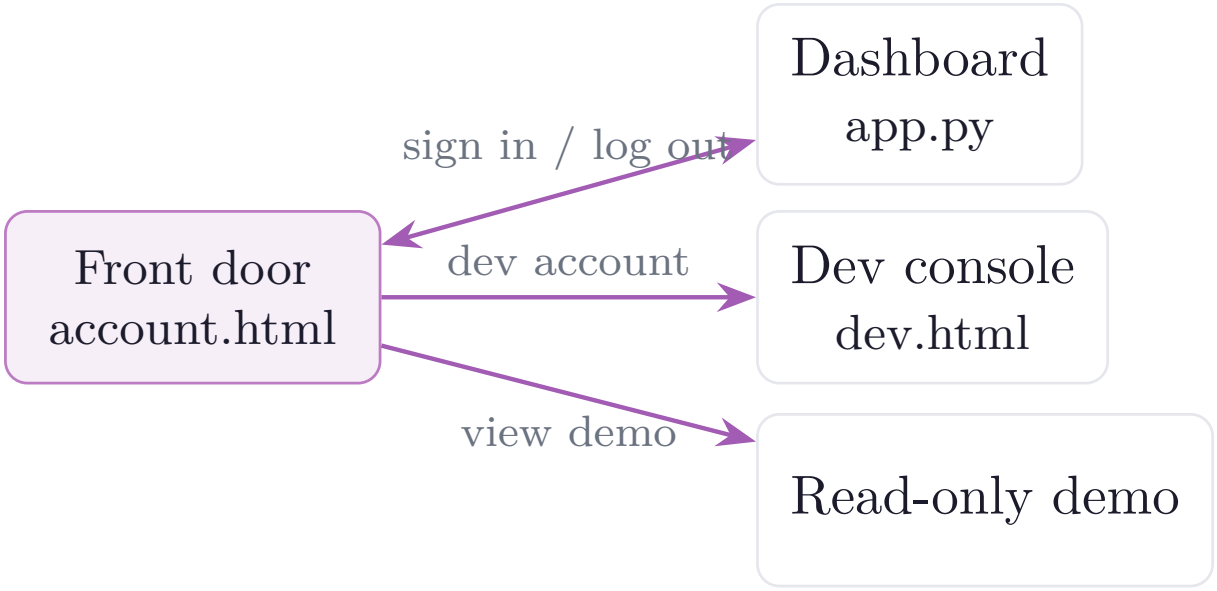


Figure 12: Dashboard page flow. The static front door is the hub for sign-in, the developer console, and the read-only demo.

10.3 Login and session

The owner logs in once on the front door with Firebase Auth, which writes the ID token into a same-origin cookie (`mtoken`). `app.py` reads that cookie client-side with the `extra-streamlit-components` cookie manager rather than the server-side cookie API, because Streamlit Community Cloud does not expose browser cookies to the server. Logged-out visitors get a sign-in gate and never see the dashboard. Logging out navigates to the front door, signs out of Firebase, and clears the cookie. No token is ever placed in a URL. A separate flag cookie marks the public read-only demo.

10.4 Code structure

`app.py` is a thin entry point that wires the pieces in order (theme, sign-in, live cards, analytics). Everything else is split into a `meowtion_dash` package so the analytics view can be edited without touching login or data-formatting code: `theme` (page config and brand header), `auth` (cookie/JWT to `(uid, token, is_demo)`), `firebase` (cached RTDB reads), `data` (the clean activity DataFrame and filter helpers), `charts` (branded Altair charts), `live` (the live current-state cards), and `weather`. The editable analytics live in `dashboard_view.py`, which receives a ready, one-row-per-event pandas DataFrame and never sees a cookie or raw Firebase JSON.

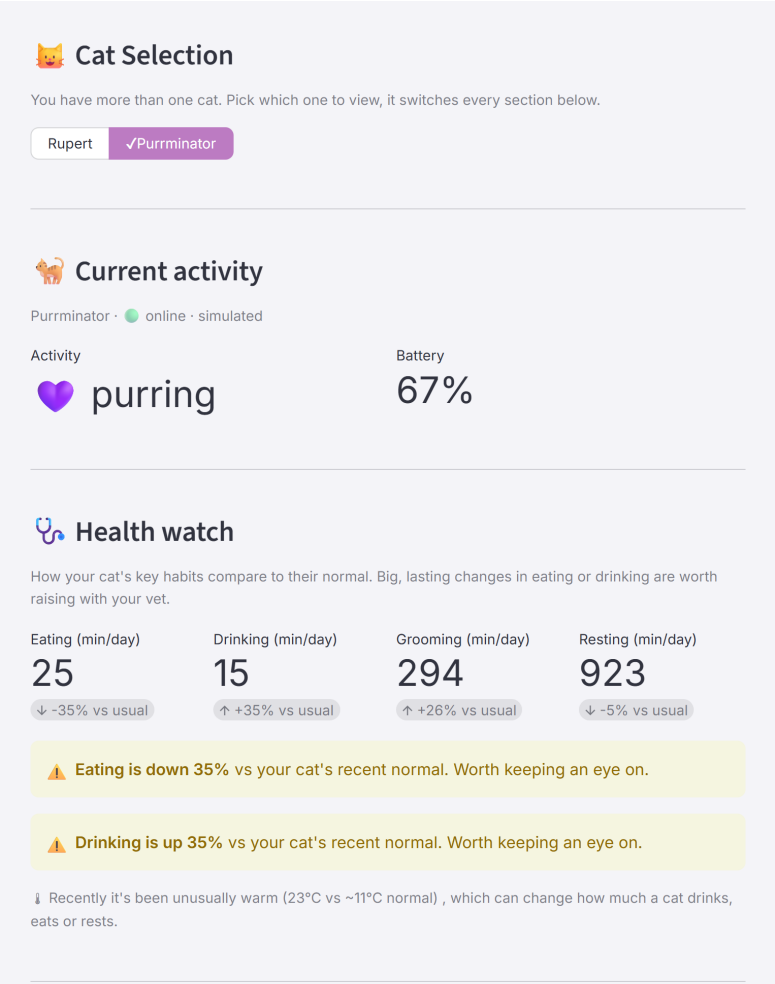


Figure 13: The dashboard (read-only demo): the collar switcher, the live current-state card, and the Health watch flagging a warm-weather rise in drinking.

10.5 Live view versus history

The two reads have very different needs and are split accordingly. The live current-state card refreshes every few seconds but needs only each cat's `current` record and its most recent events, so it issues a small per-cat read. The full activity history (potentially months of events) is fetched on a longer cache interval and reused across refreshes. This keeps the fast-refreshing live card from re-downloading the whole history on every tick, holding the page light and backend bandwidth low.

10.6 Health analytics

The analytics view is built around health, not just a live read-out. A *health watch* compares each key habit (eating, drinking, grooming, resting) over recent days against the cat's own longer-run baseline, and flags a sustained change with a plain-language, vet-oriented note rather than a diagnosis. Two design choices keep it honest. It compares whole days only, since events are logged when an action ends and the current day is always partial. And it pairs a flagged change with local weather context, so a warm-spell rise in drinking is presented with its likely cause rather than as an alarm. Below the health watch, a calendar date picker selects a single day or a date range; the view auto-derives the granularity (a single day, or multiple days) from the chosen span. It then shows a *time per activity* bar chart (total minutes per behaviour over the selected window) above a rows-of-days timeline, in which each event is a coloured bar drawn at

the time of day it occurred (y-axis: calendar day; x-axis: time of day). The timeline supports horizontal zoom (scroll or drag) with a reset-view control, and its x-axis spans the data's actual time range rather than a fixed 24 hours. Colours are assigned from a colourblind-safe palette and shared between the activity filter buttons (outline when off, filled when on) and both charts, and everything is rendered on a transparent background so it sits on the page's brand canvas.

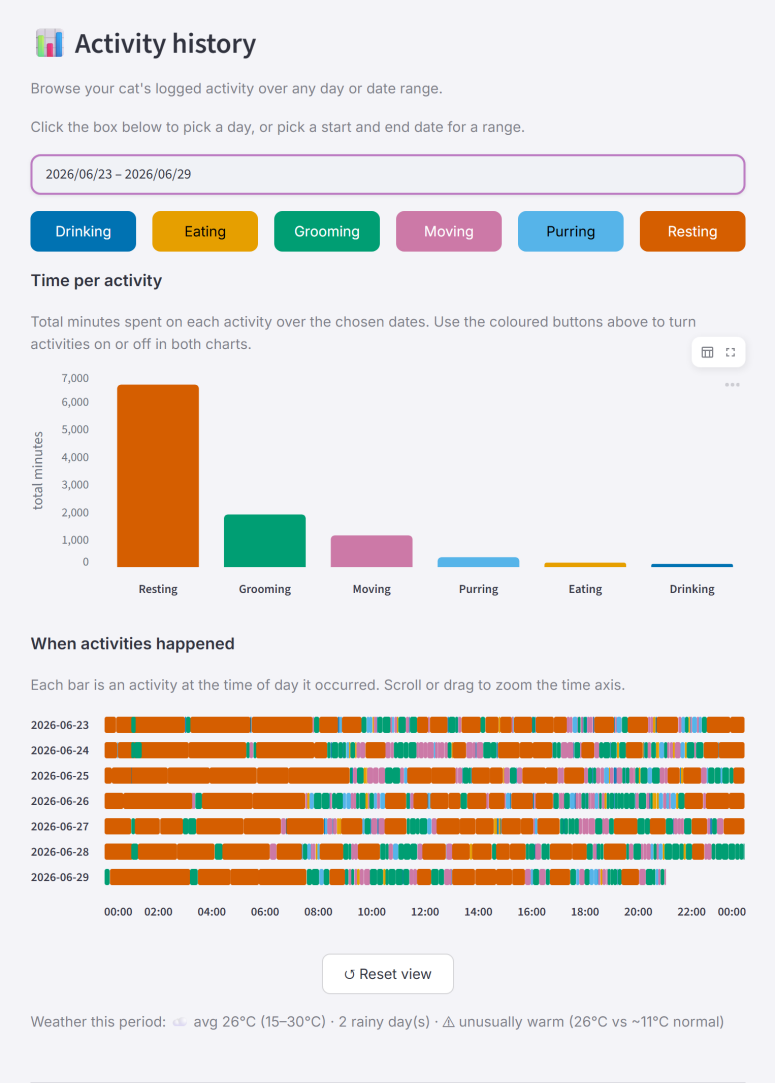


Figure 14: The history view (read-only demo): the calendar date picker, the per-activity time totals, and the zoomable rows-of-days activity timeline, with weather context.

10.7 Developer console

`dev.html` is visible only to developer accounts and is the data-collection and training control panel: live per-station capture status (signal, recording state, collar battery); the Training/Production mode switch; the editable list of actions to recognise; the labelled-clip review with waveform trimming; dataset statistics and a motion-variety measure to gauge whether a class has enough varied examples; and the button that triggers cloud training. It also offers a full-screen, phone-style capture view with large action buttons for live labelling at the bowl.

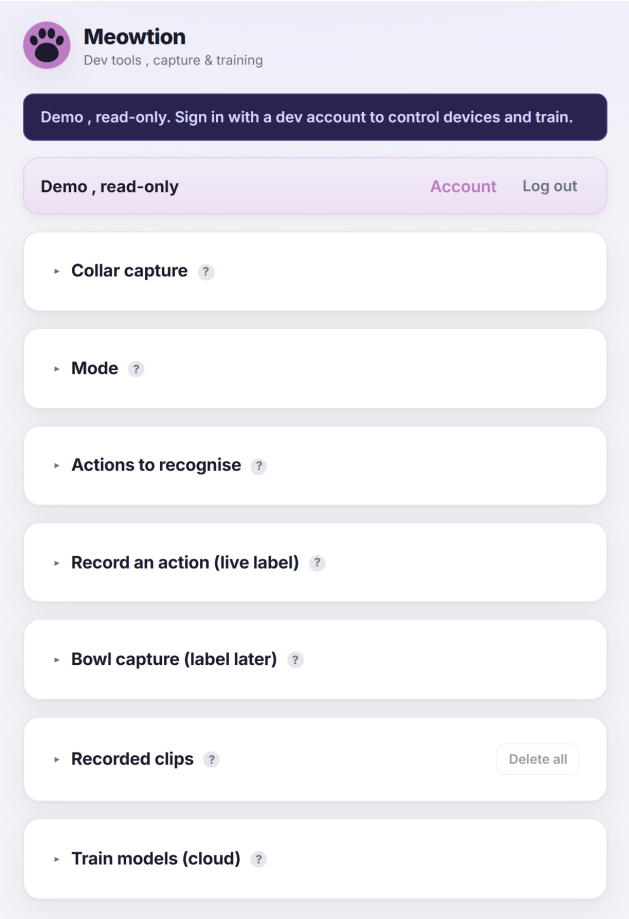


Figure 15: The developer console (read-only demo): the capture, mode, actions, clip-review, and cloud-training sections (cards collapsed by default).

10.8 Read-only demo

A public demo lets anyone explore a populated product without an account. It reads the designated demo owner's data, which the security rules make world-readable, but every write control and function trigger is disabled and the dev console is available only in a read-only variant. The demo is read-only because the backend rules enforce it, not because the interface hides the controls (section 13).

11 User interface walkthrough

This section is a guided tour of the two interactive surfaces an operator uses, the sign-in front door and the developer console. It complements the implementation in section 10 by showing each screen and what every control does. All screenshots are from the public read-only demo, so live device controls appear disabled and the private training audio is not shown.

11.1 Signing in and accounts

Figure 16 is the front door. An owner can sign in with email and password, or with **Continue with Google**. **Create one** opens registration, which records the owner's consent against the current privacy policy and sends a verification email; **Forgot password?** sends a reset link; and **View the demo (read-only)** enters the public demo with no account. Once signed in, the account page is where the owner pairs a station and registers or removes collars (the onboarding

flow is section 3), and exercises the data-protection controls: exporting everything stored about their cats, or deleting the account and all its data (section 13). A linked privacy policy page states what is collected and why.

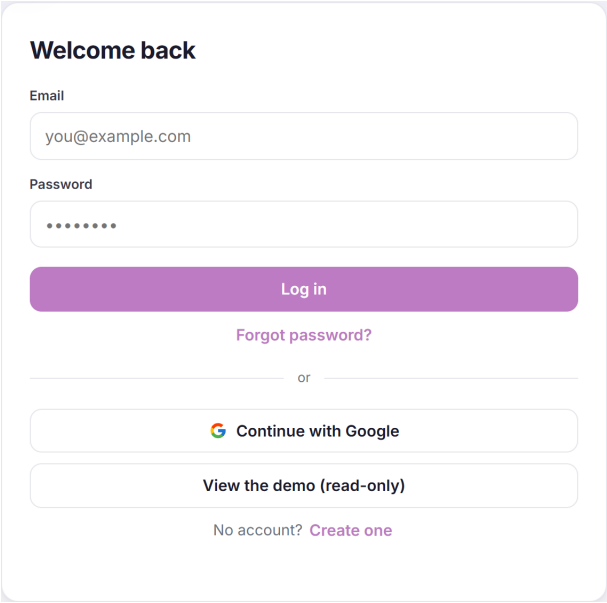


Figure 16: The sign-in front door: email/password, Google sign-in, the read-only demo, account creation, and password reset.

11.2 App navigation

Figure 17 shows the journey through the screens. The front door leads to the dashboard (by signing in, or read-only via the demo). From the dashboard a developer account can open the console, and any owner can reach the account page or log out. The console hosts the training loop, capture, label, train, then switch to production, after which detections appear back on the dashboard.

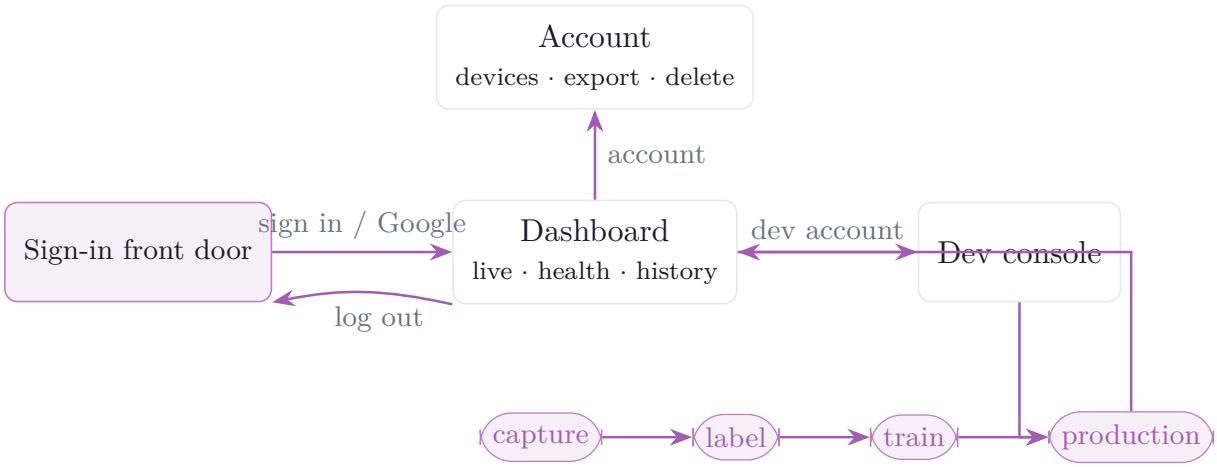


Figure 17: The user journey: front door to dashboard (or demo), the dashboard's branches to the account page and developer console, and the console's capture → label → train → production loop that feeds detections back to the dashboard.

11.3 The developer console

The developer console is where training data is collected and models are trained (sections 8 and 10). It is organised as collapsible cards, each with a ? help popover, described below in order.

Collar capture (fig. 18). Live per-station status: which station and the collar in range, the signal strength (RSSI) and whether the collar is within the capture threshold, the station's power source, the collar's battery, how many collars are registered, and how long since the station was last heard. The operator uses this to confirm a collar is close enough to record before starting a capture.

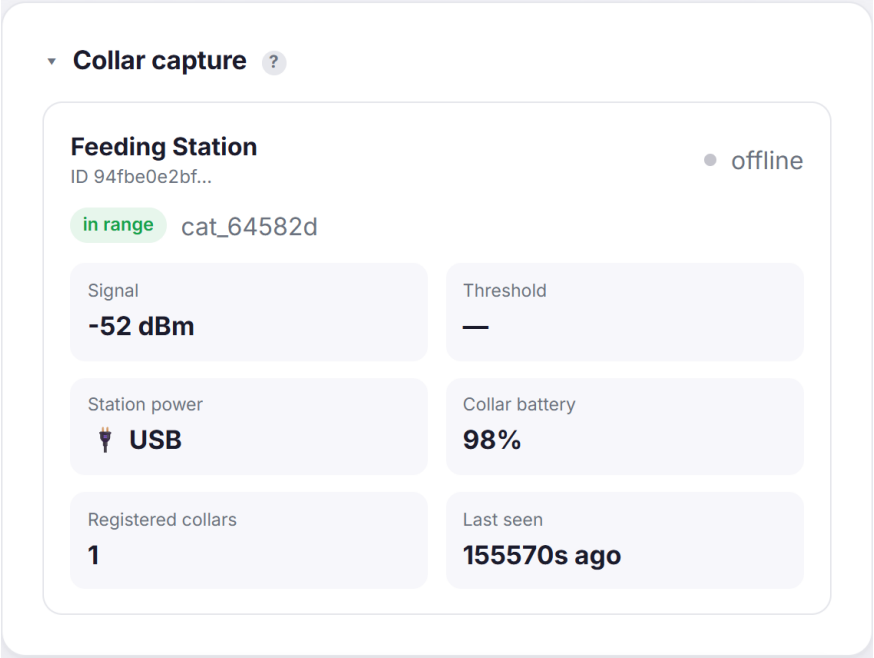


Figure 18: Collar capture: live station and collar status.

Mode (fig. 19). A single switch between **Training** (the station records and uploads clips) and **Production** (the station stops recording and only relays the collar's on-device classification). This is the mode referenced throughout section 3.

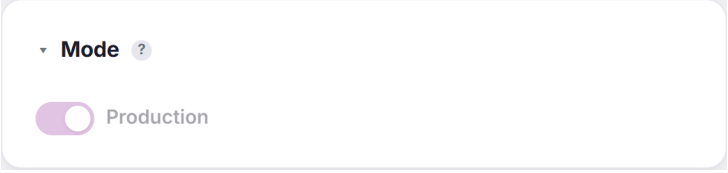


Figure 19: Mode: the Training / Production switch.

Actions to recognise (fig. 20). The editable list of behaviours the system should learn (here eat, drink, resting, moving). Adding or removing an action changes both the labels offered when reviewing clips and the classes the cloud trainer learns, which makes the pipeline label-agnostic (section 8).

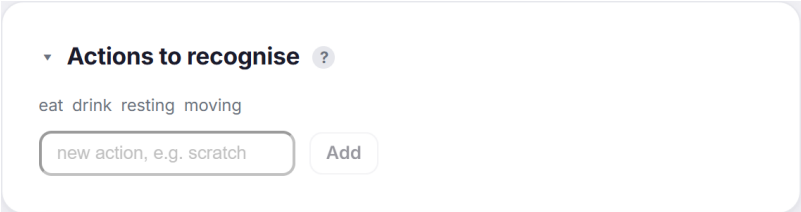


Figure 20: Actions to recognise: the owner-defined label set.

Record an action / live label (fig. 21). The fastest way to gather data: pick a clip length, press the button for what the cat is doing *now*, and the station records a clip at any distance and auto-labels it, so there is no labelling afterwards. A **Full screen** button gives a large, phone-style set of buttons for use at the bowl. A warning appears when the station is offline (as shown), since clips cannot be captured without it.

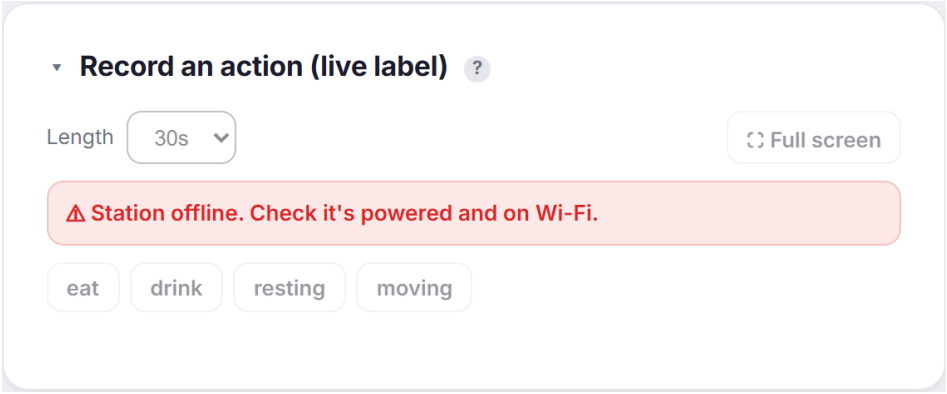


Figure 21: Record an action (live label): length, category buttons, full-screen mode, and the station-offline warning.

Bowl capture / label later (fig. 22). The alternative capture mode: with it on, the station records back-to-back whenever a registered collar is within a configurable signal-strength gate (with a dwell before it connects), and the clips are labelled afterwards in the clip review. It suits behaviours that happen reliably at the bowl.

Recorded clips (fig. 23). The dataset, and the richest card. Clips are grouped by **day** and by **session** (a run of captures within the same few minutes), so a recording run stays together. A per-label tally shows how many clips and sessions each action has, with a bar toward a recommended minimum, so the operator sees at a glance which classes need more data. **Analyse motion variety** scores how varied a label's samples are, a proxy for training quality (too-similar samples generalise poorly). Each clip row carries a play control, a label dropdown, a **Motion** badge when time-aligned IMU data is present, its time and length, and a delete control. Clicking a clip expands it to a scrubber with the *audio waveform and the motion (accelerometer and gyroscope) traces drawn time-aligned beneath it*, plus draggable trim handles, so a sample can be auditioned against its movement and tidied before training (fig. 24). This expanded view reads the clip's private audio and motion from Cloud Storage, so it is shown from a signed-in account rather than the read-only demo.

Train models (cloud) (fig. 25). **Retrain models** triggers the `train` Cloud Function on the labelled clips (section 8). The line below reports live status, the current model version, and

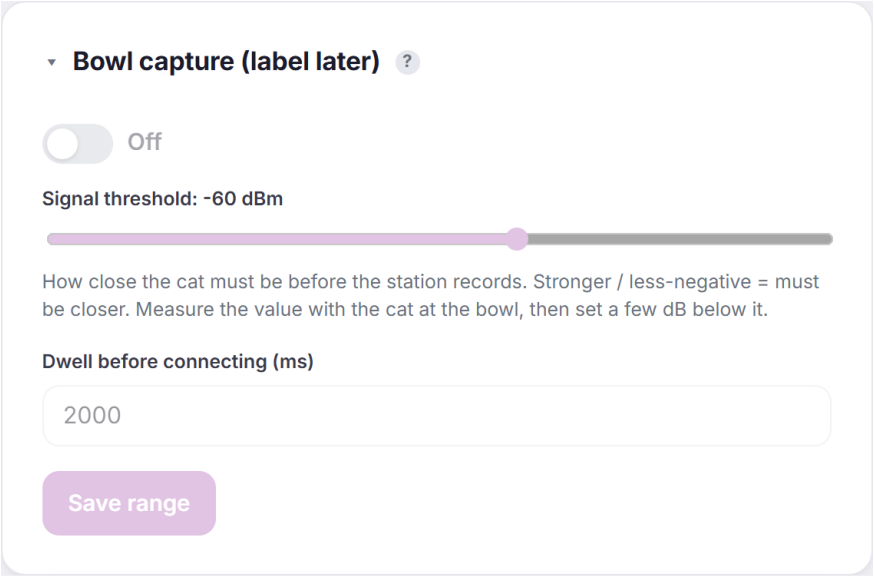


Figure 22: Bowl capture: the range gate (signal threshold and dwell) for unattended recording.

each stage's test accuracy and size (here an IMU stage and an audio stage). When training finishes, the new version is delivered to the collar over the air automatically (fig. 10).

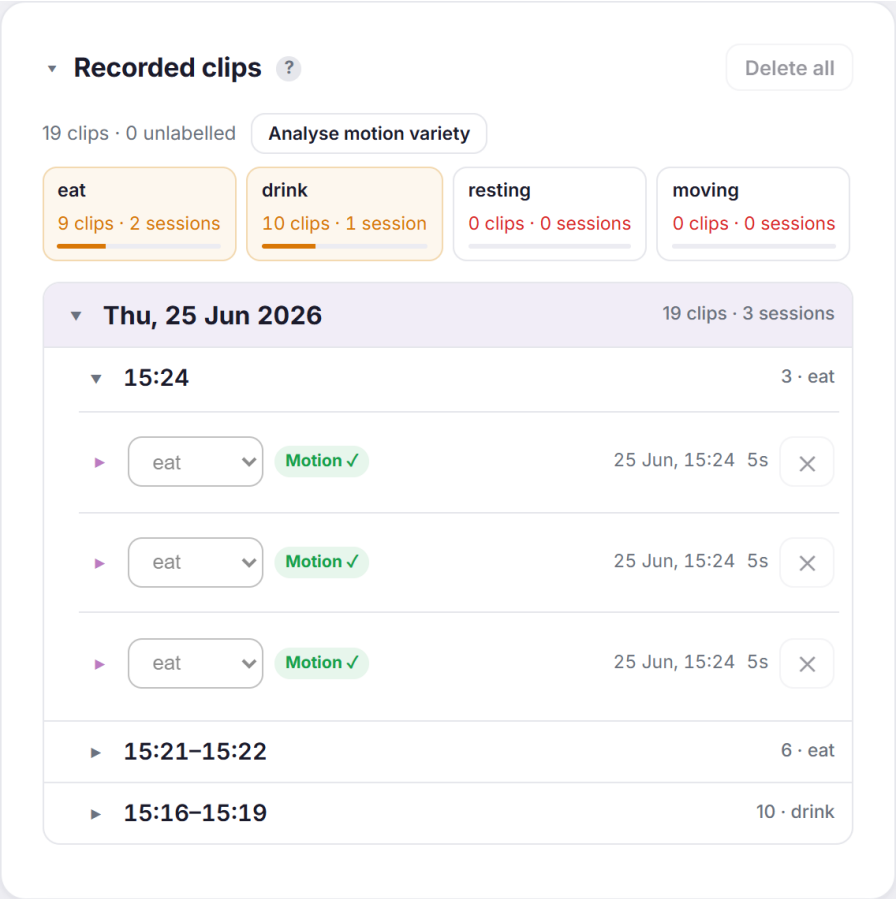


Figure 23: Recorded clips: per-label sample counts toward a minimum, the motion-variety quality check, and clips grouped by day and session, each with playback, a label, and a motion indicator.

Figure 24: A clip expanded for labelling: the audio waveform with the movement (accelerometer X/Y/Z) and rotation (gyroscope X/Y/Z) traces drawn time-aligned beneath it, plus draggable trim handles, a play control, and Save trim. The owner auditions the sound against the motion, confirms the label, and trims to the action before training.

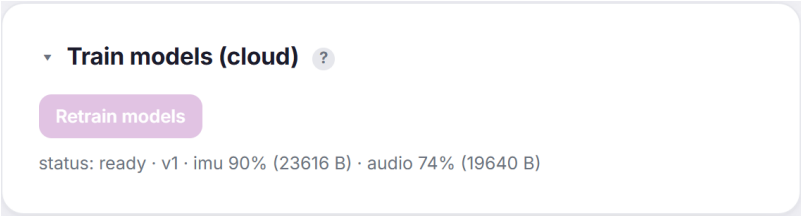


Figure 25: Train models (cloud): the retrain trigger and the live training status (version and per-stage accuracy).

PART V

Interfaces and security

12 Communication protocols

Four contracts tie the tiers together: the BLE telemetry the collar advertises, the audio clip-frame format it streams during training, the over-the-air model protocol, and the cloud HTTP it is all relayed over. The BLE wire formats are defined once in shared headers (`firmware/common/meow_protocol.h` and `meow_ota.h`) that *both* firmwares include, so the collar (Zephyr) and station (ESP-IDF) builds cannot drift out of sync.

12.1 Telemetry advertisement

In production mode the collar publishes its result in the manufacturer-specific field of its BLE advertisement, so the station can read it passively without connecting. The packet is eight bytes (table 12). The collar's BLE address identifies which collar it is, so the payload carries no identifier. The `version` byte selects how the two result bytes are read: version 1 is the simulated behaviour state machine, version 2 is real on-device classification.

Table 12: Eight-byte telemetry packet (manufacturer AD data).

Bytes	Field	Meaning
0–1	company id	`0xFFFF` (test/development identifier)
2	version	`1` = simulated state, `2` = real on-device classification
3	state / class	v1: state (0 sleep …5 groom); v2: predicted class index (`0xFE` = low-power rest, `0xFF` = unknown)
4	activity / confidence	v1: activity level 0–100; v2: confidence 0–100
5–6	steps	step count, `uint16`, little-endian
7	battery	battery level 0–100

Because the class is an index, not a name, the station and dashboard resolve it through the running model version's label list, which keeps history correct across label-set changes (section 9). The reserved class `0xFE` marks low-power rest: the rules-based activity gate advertises it while the collar sleeps, and the station records the span as a single rest episode (section 5).

12.2 Audio clip-streaming frame

While a station is subscribed in training mode, the collar streams continuously on the audio characteristic in 100 ms frames. Each frame is a packed header followed by an audio payload and a time-aligned IMU payload. The station concatenates frames and slices them into fixed-length training clips.

```
#define MEOW_CLIP_MAGIC 0x574F454Du   /* 'MEOW' little-endian, marks each frame
    start */

struct __attribute__((packed)) meow_clip_hdr {
    uint32_t magic;
    uint8_t  version;      /* 1 = 16-bit PCM audio, 2 = 8-bit G.711 mu-law */
    uint8_t  imu_axes;     /* 6 = accel x/y/z + gyro x/y/z */
    uint16_t imu_rate_hz;
```

```
    uint32_t audio_bytes;  /* audio payload size (one 100 ms frame) */
    uint32_t imu_bytes;    /* IMU payload size */
    uint32_t audio_rate;   /* output sample rate, for the station's WAV header */
};
/* frame layout:  [ meow_clip_hdr ][ audio payload ][ IMU payload ] */
```

Listing 4: Clip-frame header (`firmware/common/meow_protocol.h`).

To halve the data on the BLE link the collar encodes audio as 8-bit G.711 μ-law (`version` 2); the station decodes it back to 16-bit PCM for a normal WAV. The μ-law codec is defined inline in the same shared header, so the collar encode and station decode match. This is also the representation the model is trained and run on (section 7).

12.3 GATT services

The collar exposes two custom 128-bit GATT services that share a common Meowtion base UUID: the *audio* service for clip streaming, and the *OTA* service for model delivery. The OTA service uses three UUIDs distinguished by a `0x0b**` group, one each for its control, data, and status characteristics. Both firmwares derive these UUIDs from the same header, the collar through Zephyr's encoding macro and the station from the raw little-endian byte arrays the header also provides.

12.4 Over-the-air model protocol

Model delivery is a small framed protocol over the OTA service. The client (station) writes opcodes to the *control* characteristic, streams the model to the *data* characteristic, and the collar acknowledges on a *status* notification. A transfer is `BEGIN` (carrying the target slot, total length, and a CRC), then a stream of write-with-response `DATA` chunks (each chunk's write response is the flash-write flow control), then `END`. The collar erases the slot on `BEGIN`, writes chunks as they arrive, and on `END` re-reads the model from flash, verifies the CRC against the announced value, and reports the outcome.

```
/* CONTROL characteristic: first byte is an opcode; BEGIN is followed by
    meow_ota_begin. */
enum { MEOW_OTA_OP_BEGIN = 1, MEOW_OTA_OP_END = 2, MEOW_OTA_OP_ABORT = 3 };

struct __attribute__((packed)) meow_ota_begin {
    uint8_t  slot;       /* 0 = IMU model, 1 = audio model */
    uint32_t total_len;  /* model size in bytes */
    uint32_t crc32;      /* CRC-32/IEEE over the model blob */
};

/* STATUS characteristic (notify): the collar's acknowledgement. */
enum { MEOW_OTA_ST_READY = 1, MEOW_OTA_ST_RECEIVING = 2, MEOW_OTA_ST_OK = 3,
       MEOW_OTA_ST_ERR_SIZE = 5, MEOW_OTA_ST_ERR_CRC = 7, MEOW_OTA_ST_ERR_SEQ = 8
    };
```

Listing 5: OTA control opcodes, BEGIN payload, and status codes (`firmware/common/meow_ota.h`).

`END` verifies and stages the model but does *not* swap the live model from inside the BLE callback. Committing a model is stack-heavy, and doing it in the radio callback crash-looped the device. The swap is therefore deferred to a service call on the main thread's larger stack (section 5). That split, with the raised main stack, is what made OTA reliable on hardware. A persisted slot magic marks a slot that holds a valid committed model, so the collar can tell an empty slot from a written one across a reboot. Figure 26 shows the message exchange.

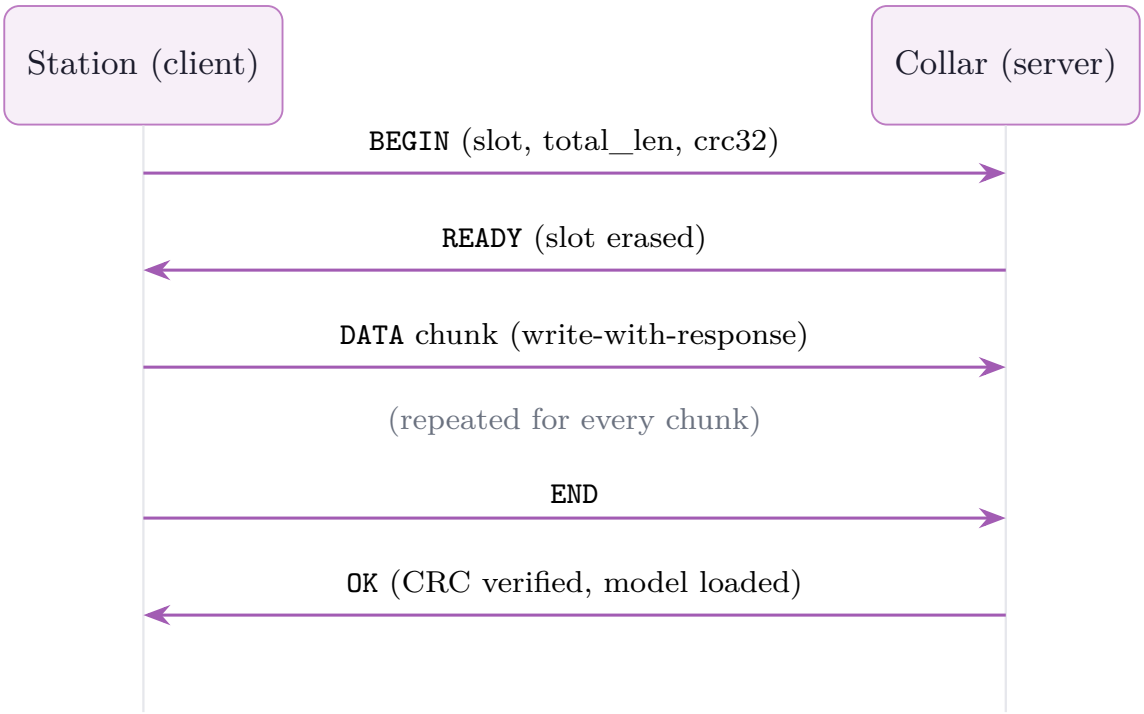


Figure 26: OTA model-push message sequence on the OTA service. The station writes `BEGIN`, streams the model in write-with-response `DATA` chunks, then writes `END`; the collar erases the slot, receives the bytes, verifies the CRC, loads the model, and acknowledges on the status characteristic.

12.5 Cloud transport

Above BLE, the station speaks ordinary HTTPS to Firebase: it writes telemetry to the Realtime Database (RTDB) through its REST interface, authorised by its device token, and POSTs training-clip bytes to the `upload_clip` Cloud Function carrying that token in an `Authorization: Bearer` header so it stays out of the URL and request logs (section 9). The dashboard is a separate client of the same RTDB. No bespoke transport or message broker is involved; the contract at this tier is just the RTDB schema and the function's request format.

13 Security and privacy

Privacy is a design property of Meowtion, not a setting bolted on afterwards. The strongest guarantees come from where work happens (on the collar) and how data is scoped (per owner, by enforced rules).

13.1 On-device audio

The microphone is used only for on-device classification. Audio is processed on the collar and discarded; it is never written to flash in normal operation and never transmitted in production mode. The collar's uplink is a classification result (a class index and a confidence), not a sensor stream. The one exception is deliberate and owner-driven: in training mode the owner chooses to capture clips to teach a behaviour, and those clips are stored privately to their own account and are never world-readable, even for the public demo.

13.2 Per-owner isolation

All of an owner's data lives under `users/<uid>/`, and the database and storage rules scope every read and write to the owning account, so owners (and their token-authenticated devices) touch only their own data. Isolation is enforced by the backend, not the client. The Storage rules go further and deny *all* direct client writes: training clips are written only by the `upload_clip` function and are readable only by their owning account, and model blobs are public-read (so the token-only station can fetch one) but writable only by the trainer.

13.3 Device tokens, not credentials

A station authenticates with a scoped, revocable per-device token rather than the owner's password, and a collar holds nothing at all. A lost or compromised device therefore cannot expose the owner's credentials, and it can be revoked simply by deleting its token from the account, with no password change and no effect on the owner's other devices. The token is carried in an `Authorization` header rather than a URL or query string, so it stays out of request logs, and the rules let an account claim only an unclaimed token or one it already owns, so a known token cannot be re-pointed to another account. Figure 27 traces the token's lifecycle.

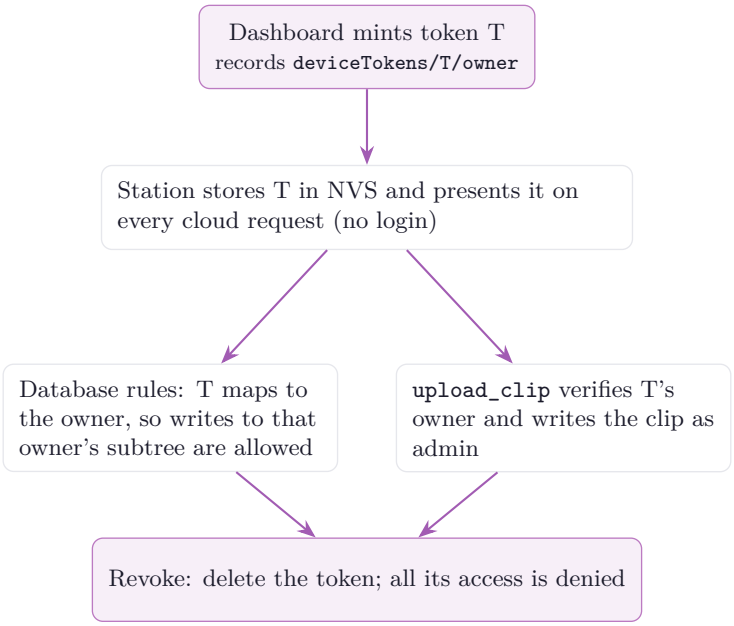


Figure 27: Credential-token lifecycle. A device is minted a scoped token that maps to its owner; the token authorises database writes and clip uploads, and deleting it revokes the device.

13.4 Enforced read-only demo

The public demo reads a single designated demo account's data, which the rules make world-readable, but that account is write-locked at the backend: the demo cannot change any data or trigger any function. The read-only behaviour is enforced by the rules, not merely hidden in the interface, so it holds even against a hand-crafted client.

13.5 Secret handling

The Firebase web API key shipped in the static front end is a public client identifier, not a secret; security is enforced by Authentication and the rules, not by hiding the key. No service-account key is present in the repository, and the functions run on their ambient service account. Device

Wi-Fi credentials are entered at runtime and never committed, and `.gitignore` guards against any local secret file. Everything required to understand and build the system is tracked; only true secrets are not.

13.6 Link encryption

The collar-to-station Bluetooth link is encrypted. Both the audio stream and the over-the-air model transfer require an encrypted connection (LE Secure Connections), established fresh on each connection so a battery collar that reboots carries no stale pairing state. A nearby device that has not paired cannot subscribe to the audio stream or write a model to the collar, so it can neither eavesdrop on the link nor overwrite the model flash; an over-the-air image is in any case committed only after a CRC-32 integrity check (section 8). Authenticating the pairing against an active man-in-the-middle, with a key provisioned over USB, is future work (section 15).

13.7 Static analysis

Every push and pull request runs CodeQL static analysis (GitHub code scanning), with findings surfaced in the repository's security tab; dependency updates are raised automatically, and the CI workflow actions are pinned to commit hashes and do not persist credentials in the checkout.

13.8 Owner data rights

The account page lets an owner export everything stored about their cats and delete their account, devices, and data. This supports the access and erasure expectations of data-protection regimes such as the UK GDPR. The bundled privacy policy is a starting point and is marked for legal review before any public launch.

PART VI

Validation and roadmap

14 Evaluation

This section reports the system's measured performance: the dataset the models were trained and evaluated on, the classification accuracy of the on-device cascade on a held-out split, the behaviour of the gated audio confirmation stage, and the on-device resource footprint. Accuracy and confusion are measured on a held-out test split of the collar's own recorded clips; the resource footprint is read from the firmware build. On-hardware timing (inference latency and over-the-air transfer) and power draw are not yet instrumented and are noted as future work (section 15).

14.1 Dataset

The models were trained and evaluated on labelled behaviour clips recorded from a single cat with the collar (section 8), each a 5 s window of μ-law audio paired with time-aligned motion. Of 191 recorded clips, 155 carry a behaviour label; the remaining 36 are unlabelled and are excluded from training and evaluation. The label set is discovered from the data rather than fixed (section 7); on this recording it is *eat*, *drink*, *resting* and *moving*.

Table 13: Dataset summary.

Property	Value
Cats	1
Capture sessions	19
Clip length	5 s
Audio sample rate	8 kHz (μ-law)
IMU sample rate	104 Hz (6-axis)
Labelled clips	155 (a further 36 are unlabelled and excluded)
Train / test split	116 / 39 clips (held-out 25 %, stratified by class)

Table 14: Labelled clips per behaviour class.

Class	Clips
Eat	94
Resting	40
Drink	11
Moving	10
Total (labelled)	155

14.2 Classification accuracy

The collar runs the confidence-gated cascade of section 7: the IMU stage classifies every window, and the audio stage confirms when the IMU result is uncertain (softmax confidence below the operating threshold of 0.75). On the held-out test split the cascade reaches an overall accuracy

of **92.3 %** (36 of 39 clips). On this dataset the IMU stage alone reaches the same 92.3 %: the gated audio stage fired but did not change the outcome (table 16 and the following subsection).

Table 15: Per-class precision / recall / F1 on the held-out split. On this data the full cascade is identical to the IMU stage alone, so a single set of figures is shown.

Class	**Precision**	**Recall**	**F1**
Eat	0.89	1.00	0.94
Resting	1.00	1.00	1.00
Drink	1.00	0.67	0.80
Moving	0.00	0.00	0.00
Overall accuracy	92.3 %		

Table 16: Confusion matrix on the held-out split (rows = actual, columns = predicted).

	Predicted			
Actual	Drink	Eat	Moving	Resting
Drink	2	1	0	0
Eat	0	24	0	0
Moving	0	2	0	0
Resting	0	0	0	10

14.3 The gated audio confirmation stage

The audio stage exists to separate behaviours that look alike to an accelerometer but differ by sound, eating and drinking being the canonical case. On the held-out split it fired on 6 of the 39 test clips (15 %), the windows where the IMU stage sat below the 0.75 confidence threshold. On this small single-cat set it changed none of those predictions and corrected no errors, so the cascade's accuracy equals the IMU stage's. The interpretation is not that the stage is redundant but that it was not needed here: the IMU model was already confident and correct on the confusable cases present in this recording, so the gated audio stage had nothing to overturn. Establishing a measured accuracy gain from the audio stage calls for a larger and more behaviourally ambiguous dataset than this first single-cat recording, and is left to future work (section 15). The value demonstrated here is the mechanism itself: a second, independent sensing modality that is consulted only when the primary stage is unsure, at no extra cost when it is confident.

14.4 Resource footprint

The collar is the binding resource constraint. The tensor arena is statically allocated and dominates the RAM cost of inference; the audio window buffer and the BLE stack (now including LE Secure Connections pairing) make up most of the rest.

14.5 Testing and verification

The system's logic is covered by automated unit tests that run in continuous integration on every change, and the integration and on-device behaviour is verified on the hardware. The security controls and their evidence are detailed separately (section 13).

Table 17: Collar resource footprint (the constraining device), read from the firmware build.

Resource	Value	Notes
RAM (static)	226 KiB of 256 KiB	Static `data+bss` with inference and the encrypted BLE link enabled (88.3 %); includes the shared tensor arena. 231 364 B
Flash (image)	337 KiB	Application image (`text+data`), excluding the model partition (63.4 %). 345 476 B
Shared tensor arena	48 KiB	One arena for both stages; only one model is resident at a time (IMU and audio flip in and out)
Model storage	256 KiB flash	Two OTA slots; the interpreter reads weights directly from this partition (XIP), with only the shared arena in RAM
IMU model size	23 736 B (≈23 KiB)	int8 `.tflite`; delivered to slot 0 over the air, not in the firmware image
Audio model size	19 760 B (≈19 KiB)	int8 `.tflite`; delivered to slot 1

- **Automated unit tests.** 80 Python tests cover the dashboard data layer and the Cloud Function's signal logic (per-window normalisation, windowing, WAV parsing, and the path-validation regexes), including property and fuzz tests that hammer the untrusted-input surface. Thirteen host-compiled C tests cover the on-device cascade (the rules-based activity gate and the confidence-gated IMU/audio gating) and the μ-law audio codec and model-input representation, the latter swept exhaustively over the whole int16 domain and checked to match the training representation. The full per-test inventory is listed in section C.
- **Continuous integration.** Every push and pull request runs the Python and C test suites and CodeQL static security analysis, with the workflow actions pinned to commit hashes.
- **On-hardware verification.** Model load-from-flash, on-device classification, the activity gate's rest/wake cycle, the encrypted collar-to-station link, and the build and backend-deploy steps were each confirmed directly on the collar and station.

14.6 Limitations and outlook

The evaluation is a first, single-cat, within-subject result and should be read as such. The dataset is small (155 labelled clips) and imbalanced: eat dominates (94 clips) while moving and drink have only 10 and 11 clips, so their per-class figures are unreliable, moving in particular scoring zero on a two-clip test slice. The held-out split is a single stratified 25 % partition, not cross-validated. On-device inference latency, over-the-air transfer time and power draw are not yet instrumented on the hardware; the resource footprint above is the measured on-device figure available now.

The progress is nonetheless encouraging, and the results point clearly at what to do next. A 92.3 % held-out accuracy from the IMU stage, learned from barely 150 clips of a single cat, is a solid baseline, and the confidence-gated cascade behaves exactly as designed: the audio stage engages precisely on the low-confidence windows and stays out of the way otherwise. That it shows no accuracy gain *yet* is a data gap, not a design flaw. With only 11 drink clips the audio model cannot yet learn the eat/drink acoustic distinction it exists to make, so it has nothing to contribute to the very cases it was built for. The clearest single lever is therefore more clips of the sparse classes, drink above all: with a balanced set the audio confirmation stage has a good prospect of delivering the disambiguation the architecture is built around, and the strong IMU baseline suggests the overall system will hold up well as the label set and the cat population

grow. Scaling the recording to more cats and behaviours, and instrumenting the on-hardware timing and power measurements, is set out as future work (section 15).

15 Future work

Meowtion already runs the whole loop on real hardware: capture, label, train, deliver over the air, classify on the collar, and surface the result live. What follows is the roadmap from that working system toward its real goal (noticing a cat's health change early), ordered so the near-term items that finish the product come first and the research-scale ones last.

From activity to health insight
The dashboard already tracks each habit and flags a large, sustained change against the cat's own baseline (section 10). The next step is the health analytics the product is really about: baselines that adapt as the cat ages, anomaly detection over a multi-day trend rather than single events, and a concise, vet-shareable summary when a habit drifts. This is the shortest path from an activity tracker to a genuine early-warning tool, and it needs no new hardware.

Richer, per-cat recognition
Extend the labelled set beyond the current eat/drink/resting/moving to scratching, litter-tray use, grooming, and play, and add a short per-cat fine-tune pass so recognition sharpens for each animal's own motion and sound. Both fall straight out of the label-agnostic pipeline (section 8).

Longer battery life
Add true hardware wake-on-motion (an IMU interrupt waking the SoC from System OFF) and tune the activity gate's thresholds and the audio path's duty cycle against the measured power budget (section 14) for the lowest possible rest current.

Multiple cats and shared stations
Several cats are already supported, each on its own collar with per-cat data and a dashboard switcher. The open problems are *model* generalisation (training on one cat and performing well on another, since the evaluation is currently single-cat and within-subject) and telling apart two cats that share a bowl or station.

Full firmware over the air
Today only models are delivered wirelessly; extend the same staged, CRC-checked mechanism (section 12) to complete firmware updates, so a deployed collar never needs a cable.

Field validation and publication
Move the accuracy study beyond a single cat with a larger, multi-household dataset, ideally paired with vet-confirmed events to test the early-warning claim directly, then write up the confidence-gated cascade as a preprint and a venue paper.

A Bill of materials

The complete parts list for one collar and one station, plus the tools and print files. There is no custom PCB; each device is a Seeed XIAO module in a 3D-printed shell. Source of truth: `hardware/README.md`.

B Data-model reference

A consolidated reference for the cloud data model introduced in section 9: the Realtime Database tree, the Cloud Storage layout, and the request format of the HTTP Cloud Functions.

Table 18: Collar parts (one per cat).

Part	Qty	Notes
Seeed XIAO nRF52840 Sense	1	SoC with BLE; on-module IMU + PDM microphone; runs inference
1S LiPo cell, 3.7 V 100 mAh, with protection (BMS)	1	Soldered to the BAT pads; protected cell required
20 mm hydrophobic PTFE filter membrane	1	Over the mic hole; passes sound, blocks water/dust
10 mm quick-release safety collar	1	Standard strap; the TPU skin mounts to it

Table 19: Station parts (one per spot).

Part	Qty	Notes
Seeed XIAO ESP32-S3	1	BLE central + Wi-Fi gateway
USB-C cable + 5 V supply	1	Mains-powered; no battery

B.1 Realtime Database tree

```
users/<uid>/
  devices/
    <stationToken>/            type: "station"
      cats/<catId>/
        current                { ver, cls, conf, steps, battery, ts, rssi,
    proximity }
        events/<startMs>        { start, durationSec, type | cls, ver }
      weather/                  { current, history }
      config/                   { modelVer, mode, captureForce, ... }
      seen/<collarId>           unregistered collars heard nearby
    <collarId>/                 type: "collar"  (registry entry; name, simulated)
  models/                       { version, status, labels[], results }
  actions/                      behaviours to recognise (dev console)
  profile/                      { consent, location }
deviceTokens/<token>/owner      maps a device token to its owning account
config/
  demoOwner                     uid backing the public read-only demo
  devAccounts/<uid>             accounts allowed to use the dev/training tools
```

Listing 6: Realtime Database layout (owner-scoped).

Events store the model `version` and class `cls` index rather than a resolved name; the dashboard maps the index to a label through that version's `labels` list, so history stays correct across label-set changes. Low-power rest spans from the activity gate are logged the same way but carry `type: "rest"` instead of a model class, so the timeline accounts for the resting period rather than leaving a gap (section 5).

The `current` record is the single live-state object per cat, written by the station. The owner dashboard reads its class and battery; the developer console reads the *same* record's station-measured `rssi` and `proximity` (`recording` / `inRange` / `approaching` / `away`). Presence and signal therefore have one source of truth rather than a separate node for the console to reconcile against the relay.

Table 20: Tools, consumables, and 3D-print files.

Item	Notes
USB-C cable	Flash and set up both boards
Filament	PETG for the rigid shells, flexible TPU for the collar skin
Water-resistant glue	Bonds the shell halves and seals against splashes
FDM 3D printer	A standard 0.4 mm nozzle is sufficient
`front-shell.stl`	Shared by collar and station (PETG)
`collar-back-shell.stl`	Collar (PETG)
`collar-silicone-skin.stl`	Collar (flexible TPU)
`station-back-shell.stl`	Station (any rigid filament)

B.2 Cloud Storage layout

```
training/<owner>/<collar>/<ts>.wav    training-clip audio   (private to owner; no
    client writes)
training/<owner>/<collar>/<ts>.imu    time-aligned motion   (private to owner)
models/<uid>/imu_model.tflite         trained IMU model     (public-read; written
    by train only)
models/<uid>/audio_model.tflite       trained audio model   (public-read; written
    by train only)
```

Listing 7: Cloud Storage layout.

B.3 Cloud Function requests

`train`
: `POST` with header `Authorization: Bearer <Firebase ID token>` of a developer account. Trains from the account's labelled clips; returns the new version and label set. Gated additionally by `config/devAccounts/<uid>`.

`upload_clip`
: `POST` the raw clip bytes with header `Authorization: Bearer <device token>` (kept out of the URL/query string). The function resolves `deviceTokens/<token>/owner` and writes the file as administrator into that owner's Storage area. No Firebase login is involved.

`simulate_now`
: `POST` with header `Authorization: Bearer <Firebase ID token>` of a developer account; manually kicks a demo-simulator run (the simulator otherwise runs on a per-minute schedule). Gated by `config/devAccounts/<uid>`.

C Automated test inventory

This appendix is the test-case specification for the project's automated tests: one filled form per test, recording its scenario, inputs, expected result and status. The suites run in continuous integration on every push and pull request (section 14). As of the run on 2026-07-01, **all 93 tests pass**: **80** Python tests (run with `pytest`) and **13** host-compiled C tests for the on-device cascade and the audio codec. Line coverage of the unit-testable core (`meowtion_dash/data.py` and the Cloud Function's `main.py`) is **100 %**. Parametrised tests are recorded as a single form whose input field lists every case. "Actual result: as expected" denotes the assertions held when the suite last ran.

C.1 Firmware tests (C, host-compiled)

Compiled and run on the host from `firmware/test` (no Zephyr, no hardware). The classifier tests link strong overrides of the model hooks (declared `weak` in `classifier.c`) and count audio-stage invocations, so every gating branch is checked.

Test case ACT-01	Unit test (host C)
Title	Motion keeps the collar awake
Component	Activity gate , `activity.c`
Preconditions	`activity_reset()` called; step `dt`=2 s.
Input / test data	100 consecutive readings at peak motion `ACT_STILL_MG`+50 (above the stillness threshold).
Procedure	Call `activity_should_rest(peak, 2)` for each reading.
Expected result	Every call returns “do not rest” , motion never drops the collar to low-power rest.
Actual result	As expected
Status	**Pass**

Test case ACT-02	Unit test (host C)
Title	Sustained stillness trips rest
Component	Activity gate , `activity.c`
Preconditions	`activity_reset()` called; step `dt`=2 s.
Input / test data	Still readings (peak 0) every 2 s, spanning the hold-off `ACT_REST_HOLDOFF_S`.
Procedure	Call `activity_should_rest(0, 2)` repeatedly across the hold-off.
Expected result	Returns “no rest” for every step before the hold-off, then “rest” once stillness is sustained past it.
Actual result	As expected
Status	**Pass**

Test case ACT-03	Unit test (host C)
Title	Motion resets the still-timer
Component	Activity gate , `activity.c`
Preconditions	`activity_reset()` called; still-timer advanced to just before the hold-off.
Input / test data	One reading at the threshold `ACT_STILL_MG`, then a still reading.
Procedure	Call `activity_should_rest(ACT_STILL_MG, 2)` then `activity_should_rest(0, 2)`.
Expected result	Does not rest immediately , the motion reset the timer, so a full hold-off is required again.
Actual result	As expected
Status	**Pass**

Test case CLF-01	Unit test (host C)
Title	Confident IMU skips audio
Component	Cascade , `classifier.c`
Preconditions	`conf_threshold`=0.75; audio confirmation enabled; audio model present.
Input / test data	IMU = (class 2, confidence 0.90); audio = (5, 0.99).
Procedure	Call `clf_classify()` for the cycle; count audio invocations.
Expected result	Audio stage not run (0 calls); result class = 2 (the confident IMU result).
Actual result	As expected
Status	**Pass**

Test case CLF-02	Unit test (host C)
Title	Unsure IMU consults audio
Component	Cascade , `classifier.c`
Preconditions	`conf_threshold`=0.75; audio confirmation enabled; audio model present.
Input / test data	IMU = (2, 0.40) below threshold; audio = (5, 0.95).
Procedure	Call `clf_classify()` for the cycle; count audio invocations.
Expected result	Audio run exactly once; result class = 5 (the more-confident audio result).
Actual result	As expected
Status	**Pass**

Test case CLF-03	Unit test (host C)
Title	Audio kept only when more confident
Component	Cascade , `classifier.c`
Preconditions	`conf_threshold`=0.75; audio confirmation enabled; audio model present.
Input / test data	IMU = (2, 0.40); audio = (5, 0.30) , less confident than the IMU.
Procedure	Call `clf_classify()` for the cycle; count audio invocations.
Expected result	Audio run once but discarded; result class = 2 (the IMU result is kept).
Actual result	As expected
Status	**Pass**

Test case CLF-04	Unit test (host C)
Title	Disabled confirmation never runs audio
Component	Cascade , `classifier.c`
Preconditions	`conf_threshold`=0.75; audio confirmation *disabled*; audio model present.
Input / test data	IMU = (2, 0.40) below threshold; audio = (5, 0.95).
Procedure	Call `clf_classify()` for the cycle; count audio invocations.
Expected result	Audio stage never runs (0 calls); result class = 2.
Actual result	As expected
Status	**Pass**

Test case CLF-05	Unit test (host C)
Title	No audio buffer skips audio
Component	Cascade , `classifier.c`
Preconditions	`conf_threshold`=0.75; audio confirmation enabled; no PCM buffer this cycle.
Input / test data	IMU = (2, 0.40) below threshold; PCM = NULL (the production IMU-only path).
Procedure	Call `clf_classify(NULL, NULL, 0)`; count audio invocations.
Expected result	Audio stage skipped (0 calls); result class = 2.
Actual result	As expected
Status	**Pass**

Test case CLF-06	Unit test (host C)
Title	No audio model skips audio
Component	Cascade , `classifier.c`
Preconditions	`conf_threshold`=0.75; audio confirmation enabled; no audio model present.
Input / test data	IMU = (2, 0.40) below threshold; audio model absent.
Procedure	Call `clf_classify()` for the cycle; count audio invocations.
Expected result	Audio stage skipped (0 calls); result class = 2.
Actual result	As expected
Status	**Pass**

C.2 Audio codec and representation tests (C, host-compiled)

From `firmware/test/test_audio_codec.c`, over the header-only `G.711` µ-law codec (`meow_protocol.h`) and the model-input transform (`audio_codec.h`). The codec invariants are swept *exhaustively* over the whole int16 domain (only 65 536 values); the transform is

fuzzed over seeded-random windows. Writing them caught and fixed a codec bug, pinned in ACO-02 below.

Test case ACO-01	Property/fuzz (host C)
Title	Codec byte round-trip is consistent
Component	µ-law codec , `meow_protocol.h`
Preconditions	None.
Input / test data	All 256 µ-law bytes.
Procedure	For each byte `b` whose `ulaw_decode(b)` is non-zero, check `ulaw_encode(ulaw_decode(b))` equals `b`.
Expected result	Holds for every byte. The two zero codes `0x7F` (-0) and `0xFF` (+0) alias to `+0` on re-encode and are exempt (standard G.711).
Actual result	As expected
Status	**Pass**

Test case ACO-02	Property/fuzz (host C)
Title	Codec preserves the sign of a non-zero sample
Component	µ-law codec , `meow_protocol.h`
Preconditions	None.
Input / test data	Every int16 of magnitude ≥ 256 (exhaustive).
Procedure	Check `ulaw_decode(ulaw_encode(s))` keeps the sign of `s` and is non-zero.
Expected result	Holds for all. Writing this caught `INT16_MIN` (-32768) encoding to near-silence , negating it as int16 overflowed back to -32768; fixed by taking the magnitude in a wider `int`.
Actual result	As expected
Status	**Pass**

Test case ACO-03	Property/fuzz (host C)
Title	Model-input transform: length and per-sample round-trip
Component	Audio representation , `audio_codec.h`
Preconditions	None.
Input / test data	4 000 seeded-random 16 kHz windows and output caps.
Procedure	Call `audio_to_model_pcm`; check the length is `floor(in/2)` clamped to the cap and each output equals the µ-law round-trip of the pairwise-averaged pair.
Expected result	Holds for every window.
Actual result	As expected
Status	**Pass**

Test case ACO-04	Differential (host C)
Title	Training wire representation equals inference representation
Component	Audio representation , `audio_codec.h`
Preconditions	None.
Input / test data	4 000 seeded-random 16 kHz windows.
Procedure	Compare `audio_to_model_pcm` (inference) against the training path (decimate 16→8 kHz, `ulaw_encode`, then the station's `ulaw_decode`).
Expected result	Identical samples , the collar trains and infers on the same representation, so there is no train/inference mismatch.
Actual result	As expected
Status	**Pass**

C.3 Cloud Function tests (Python)

From `app/firebase/functions/tests`; the Firebase SDKs are stubbed in `conftest.py`. `test_dsp.py` pins the exact representation the model trains on (and the collar reproduces at inference); the `test_helpers.py` path-validation forms are security tests.

Test case DSP-01	Unit test (pytest)
Title	Normalisation is zero-mean and bounded
Component	DSP , `main.py`
Preconditions	Firebase SDKs stubbed.
Input / test data	One channel `[1,2,3]`.
Procedure	Call `per_window_norm(x)`.
Expected result	Output mean is 0 to within `1e-5`; all values lie within `[-1,1]`.
Actual result	As expected
Status	**Pass**

Test case DSP-02	Unit test (pytest)
Title	Normalisation matches the reference formula
Component	DSP , `main.py`
Preconditions	Firebase SDKs stubbed.
Input / test data	`[0,10]`.
Procedure	Call `per_window_norm(x)`.
Expected result	Output equals `[-1,1]` exactly (mean 5 removed, then divided by max-abs 5).
Actual result	As expected
Status	**Pass**

Test case DSP-03	Unit test (pytest)
Title	Channels normalised independently
Component	DSP , `main.py`
Preconditions	Firebase SDKs stubbed.
Input / test data	Two channels `[[0,100],[4,104]]`.
Procedure	Call `per_window_norm(x)`.
Expected result	Each channel's mean is 0 independently (channels are not pooled).
Actual result	As expected
Status	**Pass**

Test case DSP-04	Unit test (pytest)
Title	Short input padded to one window
Component	DSP , `main.py`
Preconditions	Firebase SDKs stubbed.
Input / test data	3×2 array; `win`=5, `hop`=2.
Procedure	Call `windows(arr, win, hop)`.
Expected result	Exactly one window of shape `(5,2)`; the last two rows are zero-padded.
Actual result	As expected
Status	**Pass**

Test case DSP-05	Unit test (pytest)
Title	Window slides with the hop
Component	DSP , `main.py`
Preconditions	Firebase SDKs stubbed.
Input / test data	`arange(10)` reshaped `(10,1)`; `win`=4, `hop`=3.
Procedure	Call `windows(arr, win, hop)`.
Expected result	Three windows starting at indices $0, 3, 6$; each shape `(4,1)`.
Actual result	As expected
Status	**Pass**

Test case DSP-06	Unit test (pytest)
Title	Dataset windows IMU clips
Component	DSP , `main.py`
Preconditions	`LABELS`=[`eat`,`drink`].
Input / test data	One IMU clip labelled `drink`.
Procedure	Call `build_dataset([clip], "imu")`.
Expected result	`X` shape `(1, IMU_WIN, IMU_AXES)`; `y`=[1] (the label index).
Actual result	As expected
Status	**Pass**

Test case DSP-07	Unit test (pytest)
Title	Dataset windows audio clips
Component	DSP , `main.py`
Preconditions	`LABELS`=[`eat`,`drink`].
Input / test data	One audio clip labelled `eat`.
Procedure	Call `build_dataset([clip], "audio")`.
Expected result	`X` shape `(1, AUDIO_WIN, 1)`; `y`=[0].
Actual result	As expected
Status	**Pass**

Test case DSP-08	Unit test (pytest)
Title	Empty IMU clip skipped
Component	DSP , `main.py`
Preconditions	`LABELS`=[`eat`].
Input / test data	A clip with a zero-length IMU array.
Procedure	Call `build_dataset([clip], "imu")`.
Expected result	No windows produced (`len(X)`=0) , empty clips never reach training.
Actual result	As expected
Status	**Pass**

Test case VAL-01	Unit test (pytest)
Title	Valid ids accepted
Component	Path validation (security) , `main.py`
Preconditions	Firebase SDKs stubbed.
Input / test data	4 inputs: `cat_64582d`, `abc-123_XYZ`, `A`, `0`.
Procedure	Match each against `_SAFE_ID`.
Expected result	All four are accepted.
Actual result	As expected
Status	**Pass**

Test case VAL-02	Unit test (pytest)
Title	Traversal and separators rejected
Component	Path validation (security) , `main.py`
Preconditions	Firebase SDKs stubbed.
Input / test data	10 inputs: `../etc`, `a/b`, `a..b/`, `a.b`, `a b`, `a;b`, empty, `a/../b`, `..`, `a\b`.
Procedure	Match each against `_SAFE_ID`.
Expected result	All ten are rejected (no path can escape the owner's prefix).
Actual result	As expected
Status	**Pass**

Test case VAL-03	Unit test (pytest)
Title	Numeric timestamp accepted
Component	Path validation (security) , `main.py`
Preconditions	Firebase SDKs stubbed.
Input / test data	`1719600000000` (all digits).
Procedure	Match against `_SAFE_TS`.
Expected result	Accepted.
Actual result	As expected
Status	**Pass**

Test case VAL-04	Unit test (pytest)
Title	Non-numeric timestamps rejected
Component	Path validation (security) , `main.py`
Preconditions	Firebase SDKs stubbed.
Input / test data	7 inputs: `12a`, `-1`, `1.0`, `../1`, empty, `1 2`, `0x10`.
Procedure	Match each against `_SAFE_TS`.
Expected result	All seven are rejected.
Actual result	As expected
Status	**Pass**

Test case WAV-01	Unit test (pytest)
Title	WAV parser round-trips samples
Component	WAV parsing , `main.py`
Preconditions	Firebase SDKs stubbed.
Input / test data	An 8 kHz mono 16-bit WAV of `[0,1,-1,32767,-32768]`.
Procedure	Call `parse_wav(buf)`.
Expected result	Returns rate 8000 and exactly those samples (including the int16 extremes).
Actual result	As expected
Status	**Pass**

Test case WAV-02	Unit test (pytest)
Title	Sample rate read from header
Component	WAV parsing , `main.py`
Preconditions	Firebase SDKs stubbed.
Input / test data	A 16 kHz WAV.
Procedure	Call `parse_wav(buf)`.
Expected result	Parsed rate = 16000 (read from the `fmt` chunk, not assumed).
Actual result	As expected
Status	**Pass**

Test case WAV-03	Unit test (pytest)
Title	Non-WAV input rejected
Component	WAV parsing , `main.py`
Preconditions	Firebase SDKs stubbed.
Input / test data	Non-RIFF bytes.
Procedure	Call `parse_wav(buf)`.
Expected result	Raises `ValueError`.
Actual result	As expected
Status	**Pass**

Test case WAV-04	Unit test (pytest)
Title	Missing data chunk rejected
Component	WAV parsing , `main.py`
Preconditions	Firebase SDKs stubbed.
Input / test data	A WAV with a `fmt` chunk but no `data` chunk.
Procedure	Call `parse_wav(buf)`.
Expected result	Raises `ValueError`.
Actual result	As expected
Status	**Pass**

C.4 Property and fuzz tests (Python)

From `app/firebase/functions/tests/test_fuzz.py`, over the untrusted-input surface , the path validators and the WAV parser, which take fully attacker-controlled values. Each form encodes an *invariant* and throws both targeted adversarial inputs and tens of thousands of seeded-random inputs at it (stdlib `random`; no external fuzzing engine, so they run in ordinary CI). Writing them surfaced and fixed two issues, both pinned below: the id/timestamp validators accepted a trailing newline (Python's `$` also matches just before a newline, so the end is now anchored with `\Z`); and `parse_wav` raised a raw `struct.error` on a truncated `fmt` chunk instead of `ValueError`.

Test case FUZ-01	Property/fuzz (pytest)
Title	Trailing terminators rejected by the id validator
Component	Path validation (security) , `main.py`
Preconditions	Firebase SDKs stubbed.
Input / test data	Valid ids suffixed with a trailing newline, carriage return, or NUL byte (7 cases).
Procedure	Match each against `_SAFE_ID`.
Expected result	All rejected: the end is anchored with `\Z` , not `$`, so a trailing newline cannot ride into the Storage path.
Actual result	As expected
Status	**Pass**

Test case FUZ-02	Property/fuzz (pytest)
Title	Trailing terminators rejected by the timestamp validator
Component	Path validation (security) , `main.py`
Preconditions	Firebase SDKs stubbed.
Input / test data	Numeric timestamps suffixed with a trailing newline, CR, or NUL (4 cases).
Procedure	Match each against `_SAFE_TS`.
Expected result	All rejected.
Actual result	As expected
Status	**Pass**

Test case FUZ-03	Property/fuzz (pytest)
Title	Accepted ids contain only the allowed charset
Component	Path validation (security) , `main.py`
Preconditions	Firebase SDKs stubbed.
Input / test data	40 000 seeded-random strings from the allowed set plus tricky characters (newline, CR, NUL, dot, slash, backslash, space).
Procedure	For every string `_SAFE_ID` accepts, check each character is in `[A-Za-z0-9_-]`.
Expected result	No accepted string contains a disallowed character.
Actual result	As expected
Status	**Pass**

Test case FUZ-04	Property/fuzz (pytest)
Title	Accepted timestamps contain only digits
Component	Path validation (security) , `main.py`
Preconditions	Firebase SDKs stubbed.
Input / test data	40 000 seeded-random strings from digits plus tricky and letter characters.
Procedure	For every string `_SAFE_TS` accepts, check each character is a digit.
Expected result	No accepted string contains a non-digit.
Actual result	As expected
Status	**Pass**

Test case FUZ-05	Property/fuzz (pytest)
Title	Truncated fmt chunk fails cleanly
Component	WAV parsing , `main.py`
Preconditions	Firebase SDKs stubbed.
Input / test data	A WAV whose `fmt` chunk header declares a body the buffer does not contain.
Procedure	Call `parse_wav(buf)`.
Expected result	Raises `ValueError`, not a raw `struct.error` (the sample-rate read is bounds-checked).
Actual result	As expected
Status	**Pass**

Test case FUZ-06	Property/fuzz (pytest)
Title	Parser survives arbitrary bytes
Component	WAV parsing , `main.py`
Preconditions	Firebase SDKs stubbed.
Input / test data	6 000 seeded-random byte buffers, half prefixed with a valid RIFF/WAVE header.
Procedure	Call `parse_wav` on each.
Expected result	Every call returns `(int16 samples, rate)` or raises `ValueError` , never another exception, never a hang.
Actual result	As expected
Status	**Pass**

Test case FUZ-07	Property/fuzz (pytest)
Title	Parser survives adversarial chunk sizes
Component	WAV parsing , `main.py`
Preconditions	Firebase SDKs stubbed.
Input / test data	6 000 syntactically-plausible chunk streams with adversarial sizes (zero, odd, and up to 2^{31}).
Procedure	Call `parse_wav` on each.
Expected result	Returns cleanly or raises `ValueError` , the size arithmetic never overruns a read.
Actual result	As expected
Status	**Pass**

Test case FUZ-08	Property/fuzz (pytest)
Title	Normalisation stays finite and bounded under fuzz
Component	DSP , `main.py`
Preconditions	Firebase SDKs stubbed.
Input / test data	3 000 random int16 arrays of varying shape.
Procedure	Call `per_window_norm(x)`.
Expected result	Output is always finite (no NaN/inf) and bounded to about `[-1, 1]`.
Actual result	As expected
Status	**Pass**

Test case FUZ-09	Property/fuzz (pytest)
Title	Windows are always exactly one window long
Component	DSP , `main.py`
Preconditions	Firebase SDKs stubbed.
Input / test data	3 000 random `(length, win, hop)` combinations.
Procedure	Iterate `windows(arr, win, hop)`.
Expected result	Every yielded window has exactly `win` rows (short input is zero-padded, never short).
Actual result	As expected
Status	**Pass**

C.5 Dashboard data-layer tests (Python)

From `app/dashboard/tests`, over the data layer that turns raw Firebase JSON into the chart-ready frame: event parsing, the time-window drill-down, the per-day segmenting behind the timeline, and the health-watch comparison.

Test case DAT-01	Unit test (pytest)
Title	Duration formatting
Component	Data layer , `data.py`
Preconditions	None.
Input / test data	`fmt_dur(45)` and `fmt_dur(125)`.
Procedure	Call `fmt_dur`.
Expected result	Returns `45s` and `2m 05s` respectively.
Actual result	As expected
Status	**Pass**

Test case DAT-02	Unit test (pytest)
Title	Class index maps to label
Component	Data layer , `data.py`
Preconditions	`labels`=[`eat`,`drink`].
Input / test data	A `ver==2` event with `cls`=0 and `durationSec`=120.
Procedure	Call `activity_dataframe`.
Expected result	Activity is `Eating` (label 0, canonicalised); `event_duration`=2.0 min.
Actual result	As expected
Status	**Pass**

Test case DAT-03	Unit test (pytest)
Title	Rest sentinel uses type, not class
Component	Data layer , `data.py`
Preconditions	`labels`=[`eat`,`drink`].
Input / test data	A `ver==2` event with `cls`=254 (`0xFE`), `type`=rest, `durationSec`=600.
Procedure	Call `activity_dataframe`.
Expected result	Activity is `Resting` from the `type` (254 is not indexed into `labels`); 10.0 min.
Actual result	As expected
Status	**Pass**

Test case DAT-04	Unit test (pytest)
Title	Legacy v1 event uses type
Component	Data layer , `data.py`
Preconditions	None.
Input / test data	A v1 event (no `cls`) with `type`=groom.
Procedure	Call `activity_dataframe`.
Expected result	Activity is `Grooming`.
Actual result	As expected
Status	**Pass**

Test case DAT-05	Unit test (pytest)
Title	Malformed event skipped
Component	Data layer , `data.py`
Preconditions	None.
Input / test data	An event with no `start` timestamp.
Procedure	Call `activity_dataframe`.
Expected result	The row is dropped; the resulting frame is empty (no exception).
Actual result	As expected
Status	**Pass**

Test case DAT-06	Unit test (pytest)
Title	Activity filter, single and list
Component	Data layer , `data.py`
Preconditions	Frame with activities {`Eat`,`Drink`,`Rest`}.
Input / test data	Filter by `Eat`, then by [`Eat`,`Rest`].
Procedure	Call `filter_by_activity`.
Expected result	Returns {`Eat`}, then {`Eat`,`Rest`}.
Actual result	As expected
Status	**Pass**

Test case DAT-07	Unit test (pytest)
Title	Over-time sums duration
Component	Data layer , `data.py`
Preconditions	None.
Input / test data	Two same-day events of 120 s and 60 s.
Procedure	Call `over_time`.
Expected result	Per-activity total is 3.0 min.
Actual result	As expected
Status	**Pass**

Test case DSH-01	Unit test (pytest)
Title	Parses nested and standalone cats
Component	Data layer , `data.py`
Preconditions	None.
Input / test data	A device tree with a station-relayed cat and a standalone collar.
Procedure	Call `iter_cats`.
Expected result	Both are returned with name and relay resolved; the standalone's `via`=`None`.
Actual result	As expected
Status	**Pass**

Test case DSH-02	Unit test (pytest)
Title	Cat list de-duplicated
Component	Data layer , `data.py`
Preconditions	None.
Input / test data	A mixed device tree.
Procedure	Call `list_cats`.
Expected result	Returns de-duplicated `(name, id)` pairs.
Actual result	As expected
Status	**Pass**

Test case DSH-03	Unit test (pytest)
Title	Day options newest-first
Component	Data layer , `data.py`
Preconditions	None.
Input / test data	Dates 27, 28, 28 Jun.
Procedure	Call `period_options(df, "Day")`.
Expected result	Options `[2026-06-28, 2026-06-27]` , newest-first and de-duplicated.
Actual result	As expected
Status	**Pass**

Test case DSH-04	Unit test (pytest)
Title	Day and month windows
Component	Data layer , `data.py`
Preconditions	None.
Input / test data	Mixed dates; Day=2026-06-28, Month=2026-06.
Procedure	Call `filter_to_window`.
Expected result	Day keeps 2 rows; Month keeps the June rows $\{27, 28\}$.
Actual result	As expected
Status	**Pass**

Test case DSH-05	Unit test (pytest)
Title	Recent average without baseline
Component	Data layer , `data.py`
Preconditions	None.
Input / test data	3 days of eating `[10,20,30]` min.
Procedure	Call `health_signals`.
Expected result	`recent`=20.0; `baseline`=`None`; `change_pct`=`None` (too little history).
Actual result	As expected
Status	**Pass**

Test case DSH-06	Unit test (pytest)
Title	Time formatting
Component	Data layer , `data.py`
Preconditions	None.
Input / test data	An epoch-milliseconds value.
Procedure	Call `fmt_time`.
Expected result	Renders a “dd Mon · HH:MM” string.
Actual result	As expected
Status	**Pass**

Test case DSH-07	Unit test (pytest)
Title	Weekday filter
Component	Data layer , `data.py`
Preconditions	None.
Input / test data	A set {Saturday, Sunday}, and a single `Monday`.
Procedure	Call `filter_by_weekday`.
Expected result	Keeps the matching weekday rows in both cases.
Actual result	As expected
Status	**Pass**

Test case DSH-08	Unit test (pytest)
Title	Date-range filter
Component	Data layer , `data.py`
Preconditions	None.
Input / test data	A bounded range, and a start-only (open-ended) range.
Procedure	Call `filter_by_date_range`.
Expected result	Bounded keeps $\{27, 28\}$; start-only keeps $\{28\}$.
Actual result	As expected
Status	**Pass**

Test case DSH-09	Unit test (pytest)
Title	Last N calendar days
Component	Data layer , `data.py`
Preconditions	None.
Input / test data	Dates 26, 27, 28; `n`=2.
Procedure	Call `last_n_days(df, n=2)`.
Expected result	Keeps the two most recent days {27, 28}.
Actual result	As expected
Status	**Pass**

Test case DSH-10	Unit test (pytest)
Title	Habit drop versus baseline
Component	Data layer , `data.py`
Preconditions	None.
Input / test data	Eating `[100,8,8,8,999]` over 5 days.
Procedure	Call `health_signals`.
Expected result	Latest still-accruing day (999) dropped; `recent`=8.0; `baseline`=100.0; `change_pct`=-92.0.
Actual result	As expected
Status	**Pass**

Test case DSH-11	Unit test (pytest)
Title	Model labels with guards
Component	Data layer , `data.py`
Preconditions	None.
Input / test data	`models/labels` present, then `{}`, then `None`.
Procedure	Call `model_labels`.
Expected result	Returns `[eat,drink]`, then `[]`, then `[]` (no exception).
Actual result	As expected
Status	**Pass**

Test case DSH-12	Unit test (pytest)
Title	Non-dict entries skipped
Component	Data layer , `data.py`
Preconditions	None.
Input / test data	A non-dict device entry and a non-dict cat entry.
Procedure	Call `iter_cats`.
Expected result	Both are skipped; only the valid cat is returned.
Actual result	As expected
Status	**Pass**

Test case DSH-13	Unit test (pytest)
Title	Empty input and Week/Month options
Component	Data layer , `data.py`
Preconditions	None.
Input / test data	Empty input; then dates spanning two months.
Procedure	Call `period_options`.
Expected result	Empty input yields `[]`; Week/Month produce labelled options (`Week of ...`, `July 2026`/`June 2026`).
Actual result	As expected
Status	**Pass**

Test case DSH-14	Unit test (pytest)
Title	Empty input and Week window
Component	Data layer , `data.py`
Preconditions	None.
Input / test data	Empty input; then a Week window from 2026-06-29.
Procedure	Call `filter_to_window`.
Expected result	Empty input handled; the Week window keeps {2026-06-29}.
Actual result	As expected
Status	**Pass**

Test case DSH-15	Unit test (pytest)
Title	Empty input and hourly path
Component	Data layer , `data.py`
Preconditions	None.
Input / test data	Empty input; then a single-day (hourly) aggregation.
Procedure	Call `over_time`.
Expected result	Empty input yields an empty frame; the Day path sums to 2.0.
Actual result	As expected
Status	**Pass**

Test case DSH-16	Unit test (pytest)
Title	Empty-input guards
Component	Data layer , `data.py`
Preconditions	None.
Input / test data	Empty input.
Procedure	Call `last_n_days` and `health_signals`.
Expected result	Both return empty results (`[]` / empty frame) without raising.
Actual result	As expected
Status	**Pass**

Test case DSH-17	Unit test (pytest)
Title	Midnight-crossing event split per day
Component	Data layer , `data.py`
Preconditions	None.
Input / test data	Rest at 22:00 lasting 240 min (crosses midnight).
Procedure	Call `daily_segments`.
Expected result	Two rows: 22:00 to 24:00 on day 1 and 00:00 to 02:00 on day 2, mapped onto the shared time-of-day reference.
Actual result	As expected
Status	**Pass**

Test case DSH-18	Unit test (pytest)
Title	Zero-duration and empty skipped
Component	Data layer , `data.py`
Preconditions	None.
Input / test data	A zero-duration event; then empty input.
Procedure	Call `daily_segments`.
Expected result	No rows are produced in either case.
Actual result	As expected
Status	**Pass**

Test case DSH-19	Unit test (pytest)
Title	Sparse edge days trimmed
Component	Data layer , `data.py`
Preconditions	None.
Input / test data	3 days of 10 / 600 / 10 min.
Procedure	Call `trim_sparse_edge_days`.
Expected result	The thin first and last days are trimmed; the busy middle day is kept.
Actual result	As expected
Status	**Pass**

Test case DSH-20	Unit test (pytest)
Title	Sparse interior day kept
Component	Data layer , `data.py`
Preconditions	None.
Input / test data	3 days of 600 / 10 / 600 min.
Procedure	Call `trim_sparse_edge_days`.
Expected result	The thin interior day is kept (all three remain) , only edges are trimmed.
Actual result	As expected
Status	**Pass**

Test case DSH-21	Unit test (pytest)
Title	Single-day and empty handled
Component	Data layer , `data.py`
Preconditions	None.
Input / test data	A single-day frame; then an empty frame.
Procedure	Call `trim_sparse_edge_days`.
Expected result	The single-day frame is returned unchanged; the empty frame stays empty.
Actual result	As expected
Status	**Pass**

D References

[1] Judi L. Stella, Linda K. Lord and C. A. Tony Buffington. 'Sickness behaviors in response to unusual external events in healthy cats and cats with feline interstitial cystitis'. In: *Journal of the American Veterinary Medical Association* 238.1 (2011), pp. 67–73. DOI: 10.2460/javma.238.1.67.

[2] Sheilah A. Robertson. 'How do we know they hurt? Assessing acute pain in cats'. In: *Veterinary Record* 183.24 (2018), pp. 748–749. DOI: 10.1136/vr.k5384.

[3] David Bennett, Siti Mariam bt Zainal Ariffin and Pamela Johnston. 'Osteoarthritis in the cat'. In: *Journal of Feline Medicine and Surgery* 14.1 (2012), pp. 65–75. DOI: 10.1177/1098612X11432828.

[4] Atsushi Yamazaki et al. 'Utility of a novel activity monitor assessing physical activities and sleep quality in cats'. In: *PLOS ONE* 15.7 (2020), e0236795. DOI: 10.1371/journal.pone.0236795.

[5] Zephyr Project. *Zephyr RTOS Documentation*. https://docs.zephyrproject.org. Accessed 2026.

[6] Espressif Systems. *ESP-IDF Programming Guide*. https://docs.espressif.com/projects/esp-idf. Accessed 2026.

[7] Robert David et al. 'TensorFlow Lite Micro: Embedded Machine Learning for TinyML Systems'. In: *Proceedings of Machine Learning and Systems (MLSys)* 3 (2021). arXiv: 2010.08678.

[8] Liangzhen Lai, Naveen Suda and Vikas Chandra. 'CMSIS-NN: Efficient Neural Network Kernels for Arm Cortex-M CPUs'. In: *arXiv preprint* (2018). arXiv: 1801.06601.

[9] Pete Warden and Daniel Situnayake. *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*. O'Reilly Media, 2019. ISBN: 9781492052043.

[10] Google. *Firebase Documentation*. https://firebase.google.com/docs. Accessed 2026.

[11] Snowflake. *Streamlit Documentation*. https://docs.streamlit.io. Accessed 2026.